

Real-Time Network Intrusion Detection Using UFW Logs and Deep Learning with NSL-KDD Style Feature Engineering

A Submitted Thesis By

Md.Salman Saif



Department of Computer Science & Engineering
Pundra University of Science & Technology
Rangpur Road, Gokul, Bogura, Bangladesh

July, 2025

Real-Time Network Intrusion Detection Using UFW Logs and Deep Learning with NSL-KDD Style Feature Engineering

A Submitted Thesis By

Md.Salman Saif

ID:0322210105101120

Batch: 14th(Diploma)

Under the Supervision of

.....
Iffath Tanjim Moon

Lecturer

Department of Computer Science & Technology

Pundra University of Science & Technology

Rangpur Road,Gokul,Bogura,Bangladesh

July,2025

CERTIFICATION OF ORIGINALITY

We, Md. Salman Saif,, hereby declare that this submission is our original work, and to the best of our knowledge and belief, it contains no material previously published or written by another person, nor material that has been accepted for the award of any other degree or diploma from any university or institution of higher learning, except where proper acknowledgment is made within the text.

We also affirm that the intellectual content presented in this thesis is the result of our own efforts and research, although we may have received academic or technical guidance from supervisors or others regarding formatting, language, and presentation.

.....
Md.Salman Saif

CERTIFICATION OF APPROVAL

This is to certify that the thesis prepared by Md. Salman Saif, entitled “Real-Time Intrusion Detection from UFW Logs Using Machine Learning and Deep Learning Models with NSL-KDD Inspired Features”, and submitted in partial fulfillment of the requirements for the degree, complies with the regulations of the University and meets the accepted standards with respect to originality and quality.

.....
Iffath Tanjim Moon
Lecturer

ACKNOWLEDGEMENT

First and foremost, all praise and thanks to the Almighty Allah, whose blessings, mercy, and guidance have enabled us to reach this stage and complete our thesis successfully.

We would like to express our deepest gratitude and heartfelt appreciation to our honorable supervisor, Iffath Tanjim Moon, Lecturer, Department of Computer Science and Engineering, Pundra University of Science & Technology. Her unwavering support, profound knowledge, and continuous encouragement have been a cornerstone throughout the entire duration of this research. Her valuable guidance not only helped us to shape our ideas and structure the work properly but also motivated us to explore, analyze, and evaluate our topic from a deeper perspective. Her patience, generosity in feedback, and inspiring words served as a beacon during moments of challenge and doubt.

We are also immensely thankful to the entire faculty and staff of the Department of Computer Science and Engineering for providing a nurturing academic environment that enabled us to grow both intellectually and personally.

We are profoundly grateful to our parents for their constant love, endless sacrifices, and emotional strength. Their encouragement and faith in us provided the foundation for our determination and perseverance throughout this academic journey. In moments of difficulty, their unconditional support reminded us of our purpose and helped us stay focused on our goals.

We would also like to acknowledge our peers and well-wishers who supported us directly or indirectly with constructive discussions, technical advice, and moral encouragement. Their contributions, though informal, were invaluable and sincerely appreciated.

In conclusion, the successful completion of this thesis is not just a personal achievement but the result of a collective effort from all the individuals who stood by our side. We extend our sincerest gratitude to all of them. May Allah reward them all with continued success, good health, and infinite happiness.

ABSTRACT

In today's interconnected digital ecosystem, ensuring robust cybersecurity has become a critical priority due to the rising frequency and sophistication of cyber threats. Traditional Intrusion Detection Systems (IDS) often depend on deep packet inspection and static rule-based techniques, which are limited in adaptability and efficiency—especially in detecting zero-day attacks or operating in real-time resource-constrained environments. This research proposes a lightweight, real-time IDS framework that leverages structured Uncomplicated Firewall (UFW) logs generated from Linux systems and transforms them using a customized NSL-KDD-style feature engineering pipeline. The dataset was constructed from real-world UFW log entries, resulting in a cleaned and preprocessed tabular format containing over 125,000 instances and 22 attack categories. A diverse set of eleven machine learning and deep learning models was evaluated using stratified 10-fold cross-validation. Evaluation metrics included Accuracy, Precision, Recall, F1-Score, AUC, FPR, and FNR. Among the models, Decision Tree (99.33%) and Random Forest (99.36%) achieved the highest accuracy, followed by CatBoost (98.94%), LightGBM (98.89%), and XGBoost (96.93%). Deep learning models like TabTransformer (95.89%), LSTM (93.05%), and Autoencoder (89.68%) demonstrated strong capabilities in learning temporal and structural patterns. In contrast, traditional classifiers like Logistic Regression (92.09%), and Support Vector Machine (84.28%) showed moderate performance. The proposed framework demonstrates significant potential for practical deployment in SMEs, educational institutions, and edge environments. By avoiding dependency on packet payloads and focusing on behavior-level indicators extracted from logs, the system ensures enhanced scalability, privacy compliance, and efficiency. This study contributes to the evolving landscape of intelligent cybersecurity solutions by introducing a real-time, log-based IDS model grounded in modern machine learning practices and validated on real-world data.

Keywords: *Intrusion Detection System, UFW Logs, Tabular Feature Engineering, Machine Learning, Deep Learning, ,*

CONTENTS

CERTIFICATION OF ORIGINALITY	i
CERTIFICATION OF APPROVAL	ii
ACKNOWLEDGEMENT	iii
ABSTRACT	iv

CHAPTER ONE

INTRODUCTION	1
1.1 Background and Context	1
1.2 Motivation and Research Gap	2
1.3 Problem Statement	3
1.4 Research Objectives	4
1.5 Contributions of the Study	4
1.6 Literature Review	5

CHAPTER TWO

RELATED WORK	6
2.1 Signature and Packet-Based Detection Approaches	6
2.2 NSL-KDD Based Machine Learning Research	7
2.3 Log-Based Behavioral Intrusion Detection	8
2.4 Gaps in Existing Approaches	9

CHAPTER THREE

METHODOLOGY	10
3.1 Dataset Description	10
3.2 Data Preprocessing	11
3.3 Feature Engineering (NSL-KDD Inspired)	12
3.4 Model Selection and Training	13
3.5 Validation Strategy (10-Fold CV).....	14
3.6 Evaluation Metrics Used	15
3.7 Experimental Setup and Tools	16

CHAPTER FOUR

EXPERIMENTAL RESULTS	17
4.1 Model-wise Accuracy Comparison	17
4.2 ROC Curves and Confusion Matrices	19
4.3 Cross-Validation Insights	21
4.4 Visual Comparison of Evaluation Metrics	22

CHAPTER FIVE

DISCUSSION	23
5.1 Interpretation of Results	23
5.2 Strengths and Weaknesses of Models	24
5.3 Use-Case Implications and Practicality	25

CHAPTER SIX

CONCLUSION AND FUTURE WORK.....	26
6.1 Summary of Findings	26
6.2 Limitations of the Study	27
6.3 Future Research Directions	27
REFERENCES	28

CHAPTER ONE

INTRODUCTION

This research introduces a structured methodology that converts raw UFW logs into machine-readable formats using feature engineering techniques inspired by benchmark datasets like NSL-KDD. The study further evaluates the effectiveness of various ML and DL algorithms—including modern models such as TabTransformer and LSTM—for detecting anomalies. The aim is to design a lightweight, scalable, and real-world intrusion detection framework based entirely on log data.

In the following chapters, a comprehensive exploration is presented—starting with existing literature on intrusion detection and log-based monitoring, followed by detailed preprocessing steps such as log parsing and normalization. The methodology chapter explains model selection, hyperparameter tuning, and validation strategies including stratified 10-fold cross-validation. Subsequent sections present experimental results through comparative analysis of model performance metrics. Finally, the study provides an in-depth discussion of real-world applicability, system limitations, and future research opportunities for enhancing log-based IDS solutions using hybrid and adaptive learning methods.

1.1 Background and Context

In today's highly interconnected digital landscape, the frequency and complexity of cyberattacks have grown exponentially. Protecting computer systems from unauthorized access, data breaches, and malicious threats has become a top priority for organizations and individuals alike. Intrusion Detection Systems (IDS) are widely adopted to monitor and detect unusual or harmful activities within a network. Traditional IDS models often depend on deep packet inspection or static rule-based methods, which are computationally expensive and fail to detect zero-day attacks effectively.

With the advancement of artificial intelligence, machine learning (ML) and deep learning (DL) have emerged as promising alternatives in developing intelligent, adaptive IDS models. Simultaneously, system-generated logs, such as those from the Uncomplicated Firewall (UFW) in Linux environments, offer a rich source of behavioral data for intrusion detection. However, effectively transforming unstructured log data into a structured format suitable for ML/DL remains a challenge. This study addresses that challenge by combining UFW logs with NSL-KDD-inspired feature engineering to develop a scalable, real-time intrusion detection solution.

1.2 Motivation and Research Gap

Most existing IDS solutions focus on either packet-level inspection or predefined static rules, often neglecting host-level logs such as those generated by firewalls. Logs, especially from Uncomplicated Firewall (UFW) in Linux, offer sequential and behavior-centric insights into network activities. These logs are lightweight, privacy-compliant, and readily available in most Linux systems, making them ideal for real-time intrusion detection in low-resource settings. Despite this, there exists a significant research gap: structured log-based ML/DL models remain underexplored, particularly those leveraging modern architectures like TabTransformer and LSTM. Moreover, limited efforts have been made to design reusable feature engineering pipelines for transforming raw logs into structured, learnable datasets suitable for advanced anomaly detection. This gap highlights the need for practical, scalable, and real-world log-based IDS frameworks.

1.3 Problem Statement

Traditional IDS methods are ill-equipped to handle the dynamic nature of modern cybersecurity threats. Specifically, they suffer from:

- Inability to detect novel or zero-day attacks due to reliance on predefined rules or signatures
 - Poor scalability and high computational cost, especially in packet inspection-based methods
 - Overdependence on outdated or synthetic datasets, limiting real-world applicability
 - Lack of structured use of system-generated logs for behavior-based anomaly detection
- This research addresses the challenge of building a lightweight and real-time IDS system that utilizes Linux UFW logs with structured feature transformation to enable accurate and fast intrusion detection.

1.4 Research Objectives

This research aims to design an intelligent and efficient intrusion detection framework by leveraging structured log data and advanced machine learning techniques. The key objectives are:

- To develop a log-to-tabular feature engineering pipeline inspired by the NSL-KDD dataset structure
- To transform real UFW logs into a structured dataset suitable for supervised ML/DL modeling
- To compare the performance of traditional ML and advanced DL models in detecting anomalous behavior
- To evaluate model performance using rigorous cross-validation and multiple metrics (Accuracy, AUC, F1-score, etc.)
- To propose a lightweight, deployable IDS framework for resource-constrained environments

1.5 Contributions of the Study

This research introduces a comprehensive and practical approach to intrusion detection by leveraging real-world firewall logs and advanced learning models. The major contributions of this study are:

- Introduction of a structured log-based dataset derived from real-world UFW logs
- Design and implementation of a feature engineering pipeline aligning with the NSL-KDD schema
- Comparative analysis of eleven ML/DL models including TabTransformer, LSTM, and CatBoost
- Empirical validation through 10-fold cross-validation on over 125,000 log entries
- Demonstration of model scalability, efficiency, and accuracy in real-time environments

1.6 Literature Review

The rapid evolution of cyber threats has exposed the shortcomings of traditional Intrusion Detection Systems (IDS), especially those based on rule-matching or signature-based mechanisms. Tools such as Snort and Suricata are representative of this paradigm and, although they offer high precision in recognizing known attacks, they perform poorly when confronted with novel or modified threats. Their dependency on signature databases renders them ineffective against zero-day or polymorphic attacks, and the reliance on deep packet inspection incurs significant computational overhead, limiting their applicability in real-time environments, particularly on low-resource systems [1], [2].

To overcome these limitations, a shift toward machine learning (ML) and deep learning (DL) based IDS solutions has gained momentum. Datasets like NSL-KDD, which was developed to address the flaws of the outdated KDD'99 dataset—such as redundant records and class imbalance—have become widely adopted benchmarks for evaluating ML-based intrusion detection approaches [3]. Several algorithms, including Decision Tree, Random Forest, and Support Vector Machines, have achieved over 95% accuracy on these datasets [4]. Furthermore, unsupervised models like autoencoders have been used to detect anomalies without relying on labeled data, enabling detection in situations where labeling attack data is infeasible [5].

However, a critical shortcoming of existing ML-based IDS lies in their reliance on synthetic or outdated datasets that do not reflect modern network environments, including encrypted communications, dynamic port usage, or advanced persistent threats (APTs). This severely limits the practical effectiveness of such models in real-world settings. Moreover, many prior studies failed to incorporate log-based behavioral data, which offers richer temporal and contextual information than packet-level analysis [6].

Recent investigations have explored the potential of system-generated logs, such as Linux UFW (Uncomplicated Firewall) entries, for detecting anomalies. Studies by Patil & Meshram [7] and Tamilarasi et al. [8] have shown that system logs, when processed with ML classifiers like SVM and Decision Tree, can aid in recognizing malicious patterns. However, these studies generally

lacked comprehensive feature engineering, and few utilized deep learning architectures, resulting in limited capacity to detect complex or evolving attacks.

Another gap in the literature is the underutilization of advanced DL models like Long Short-Term Memory (LSTM) and TabTransformer. LSTM networks are particularly well-suited for handling sequential dependencies in time-stamped logs, while TabTransformers excel at modeling tabular data with categorical and numerical features. Despite their promise, these models have rarely been applied to structured firewall log data in a complete ML/DL pipeline involving preprocessing, normalization, and robust evaluation.

Research Contribution

To bridge these gaps, the present study proposes a hybrid intrusion detection framework that combines real-time UFW firewall logs with a custom feature engineering pipeline inspired by NSL-KDD. The dataset is transformed from raw unstructured logs into structured, machine-readable tabular format, enabling both traditional and deep learning classifiers to be trained efficiently.

Key contributions include:

- Integration of real-world Linux firewall logs with modern deep learning models for practical IDS deployment.
- Comparative evaluation of a wide range of classifiers, including Autoencoder, LSTM, and TabTransformer, under 10-fold stratified cross-validation for robustness.
- Incorporation of false positive rate (FPR) and false negative rate (FNR) as additional performance indicators for practical deployment in safety-critical systems.
- Emphasis on deployability for resource-constrained environments, such as edge devices and SMEs.

By aligning real-world log data with advanced modeling techniques, this study advances the current landscape of IDS research and highlights the practical utility of behavior-based detection methods in modern network security.

CHAPTER TWO

RELATED WORK

In recent years, numerous approaches have been proposed to improve intrusion detection systems, ranging from traditional rule-based techniques to modern machine learning models. This chapter outlines the strengths and limitations of key methodologies relevant to this study.

2.1 Signature and Packet-Based Detection Approaches

Signature-based Intrusion Detection Systems (IDS), such as Snort and Suricata, operate by matching observed traffic patterns against a database of known attack signatures. These systems are known for their high accuracy in detecting known threats and have been widely adopted in both commercial and open-source environments (Roesch, 1999). However, a critical limitation of these methods is their inability to detect unknown or zero-day attacks, as the detection process is strictly bound to predefined rules.

Packet inspection-based methods, on the other hand, analyze the contents of individual packets for suspicious activity. While effective in some scenarios, this method is computationally expensive and inefficient in encrypted environments, where payload data is inaccessible. In high-throughput or real-time settings, these methods introduce latency and scalability bottlenecks, making them less suitable for modern decentralized and cloud-native architectures (Ahmed et al., 2016).

2.2 NSL-KDD Based Machine Learning Research

The NSL-KDD dataset, proposed by Tavallaee et al. (2009), addressed several issues in the original KDD'99 dataset such as duplicate records and skewed distributions. This has made it a benchmark for evaluating ML-based IDS models. Numerous researchers have applied classical ML models—including Decision Tree, Random Forest, Support Vector Machine—achieving accuracy rates above 95% in most cases (Revathi & Malathi, 2013).

In the deep learning domain, Shone et al. (2018) proposed an unsupervised autoencoder-based IDS using NSL-KDD, showing promising results in anomaly detection. Despite its popularity, NSL-KDD has several limitations—it is a synthetic dataset, lacks real-world firewall behaviors, and fails to represent modern cyber threats such as botnets, DDoS, or encrypted traffic patterns, reducing its practical applicability for current systems.

2.3 Log-Based Behavioral Intrusion Detection

In response to the shortcomings of packet and signature-based methods, recent research has focused on log-based behavioral intrusion detection, which leverages structured logs generated by systems such as firewalls, authentication services, and system processes. This approach has the benefit of working without needing access to encrypted payloads, ensuring better privacy compliance and real-time efficiency.

Uncomplicated Firewall (UFW), a widely used firewall system in Linux, generates logs containing temporal and protocol-based behaviors that reflect real-world user and system activities. Previous work by Patil & Meshram (2019) used SVM and Decision Tree classifiers on system logs, while Tamilarasi et al. (2021) explored Linux audit logs to detect behavioral anomalies. However, these studies lacked structured feature engineering pipelines and often used limited or manually extracted features, resulting in suboptimal model performance.

Moreover, most research in this domain fails to utilize modern deep learning architectures such as LSTM (which handles temporal dependencies) or TabTransformer (which models categorical/tabular data). This reveals a gap in effectively modeling real-time log data using state-of-the-art techniques.

2.4 Gaps in Existing Approaches

Although intrusion detection research has advanced substantially over the years, several critical limitations continue to hinder the effectiveness and practicality of many proposed solutions. The major gaps identified in the current body of work include:

- Heavy reliance on synthetic datasets like NSL-KDD, which are outdated and not reflective of real-world log behavior.
- Limited adoption of log-based intrusion detection using structured firewall data such as UFW.
- Scarce usage of advanced deep learning models like TabTransformer or LSTM for log analysis.
- Lack of end-to-end pipelines that convert raw logs into structured features inspired by NSL-KDD and test across diverse ML and DL models.
- Absence of emphasis on lightweight and deployable IDS frameworks suitable for edge environments or SMEs.

This research addresses these gaps by proposing a real-time, log-based intrusion detection system using real UFW logs and a structured feature transformation pipeline, benchmarked using a broad spectrum of classical and deep learning models.

METHODOLOGY

To effectively design a log-based intrusion detection system, this chapter outlines the dataset preparation process, feature extraction techniques, and the machine learning and deep learning models employed in the study. A step-by-step approach was adopted to ensure the framework remains scalable, reproducible, and suitable for real-world deployment.

3.1.1 Source of the Dataset

The dataset employed in this research was generated from real-time Uncomplicated Firewall (UFW) logs collected from a Ubuntu 22.04 LTS-based virtual private server (VPS) hosted on DigitalOcean. The logging period spanned 31 consecutive days, ensuring inclusion of diverse network behaviors across weekdays and weekends.

The VPS was configured to run multiple standard services such as OpenSSH (port 22), Apache2 web server (ports 80/443), and MySQL (port 3306) to emulate real-world traffic patterns. Routine administrative tasks, software updates, and authorized SSH sessions from trusted IP addresses formed the benign (normal) traffic class.

To simulate malicious behavior, controlled attack scenarios were introduced in a sandboxed and monitored environment. Several well-known penetration testing and traffic simulation tools were used, including:

- Nmap – for various port scanning techniques (SYN scan, UDP scan, OS detection)
- Hping3 – to generate custom TCP/UDP packets and perform SYN floods
- Metasploit Framework – for executing exploit modules targeting known vulnerabilities

All attack attempts were launched from separate client machines located outside the VPS network to ensure authenticity in log traces. Each log entry captured by UFW contained rich metadata such as:

- Timestamp of event
- Action taken (ALLOW or BLOCK)
- Protocol type (TCP/UDP/ICMP)
- Source and Destination IP and Port
- Network Interface and MAC Address
- Packet size and flags (e.g., SYN, ACK)

Raw log files were continuously archived and later parsed into structured tabular format for further processing and labeling. This hybrid strategy of live monitoring and controlled adversarial testing ensured that the dataset reflected both legitimate user behavior and diverse real-world attack signatures, making it highly suitable for building and benchmarking an intrusion detection model based on behavioral patterns rather than packet payloads.

3.1.2 Sample Log Structure

The Uncomplicated Firewall (UFW) on Linux systems generates log entries in a structured format that reflects real-time network activity. Each log line represents a snapshot of a network event and typically includes metadata about the action taken, the protocol used, and the connection details such as source and destination addresses and ports.

A typical raw UFW log entry appears as follows:

```
Jun 21 10:15:24 ubuntu kernel: [UFW BLOCK] IN=eth0 OUT= MAC=00:0c:29:bd:6c:32:00:50:56:e7:1d:3b:08:00 SRC=192.168.0.2 DST=192.168.0.10 LEN=52 TOS=0x00 PREC=0x00 TTL=64 ID=54321 DF PROTO=TCP SPT=445 DPT=55846 WINDOW=8192 RES=0x00 SYN URGP=0

Jun 21 10:15:25 ubuntu kernel: [UFW ALLOW] IN=eth0 OUT= MAC=00:0c:29:4a:73:fa SRC=192.168.0.3 DST=192.168.0.11 LEN=60 TOS=0x00 PREC=0x00 TTL=128 ID=12345 DF PROTO=UDP SPT=53 DPT=5454 LEN=32

Jun 21 10:15:26 ubuntu kernel: [UFW BLOCK] IN=eth1 OUT= MAC=08:00:27:00:44:aa SRC=10.0.2.15 DST=10.0.2.5 LEN=64 TOS=0x00 PREC=0x00 TTL=63 ID=3333 DF PROTO=ICMP TYPE=8 CODE=0 ID=456 SEQ=1

Jun 21 10:15:27 ubuntu kernel: [UFW ALLOW] IN=lo OUT= MAC= SRC=127.0.0.1 DST=127.0.0.1 LEN=48 TOS=0x00 PREC=0x00 TTL=64 ID=1 DF PROTO=TCP SPT=8000 DPT=8000 WINDOW=65535 RES=0x00 SYN URGP=0

Jun 21 10:15:28 ubuntu kernel: [UFW BLOCK] IN=eth0 OUT= MAC=00:1a:2b:3c:4d:5e SRC=172.16.1.10 DST=172.16.1.1 LEN=40 TOS=0x00 PREC=0x00 TTL=50 ID=56789 DF PROTO=TCP SPT=23 DPT=22 WINDOW=0 RES=0x00 RST URGP=0

Jun 21 10:15:29 ubuntu kernel: [UFW ALLOW] IN=eth0 OUT= MAC=00:0c:29:55:66:77 SRC=192.168.1.5 DST=192.168.1.100 LEN=60 TOS=0x00 PREC=0x00 TTL=64 ID=20000 DF PROTO=UDP SPT=67 DPT=68 LEN=32

Jun 21 10:15:30 ubuntu kernel: [UFW BLOCK] IN=eth0 OUT= MAC=00:0c:29:dd:ee:ff SRC=10.10.10.10 DST=192.168.1.10 LEN=52 TOS=0x00 PREC=0x00 TTL=64 ID=9999 DF PROTO=TCP SPT=21 DPT=8080 WINDOW=8192 RES=0x00 SYN URGP=0

Jun 21 10:15:31 ubuntu kernel: [UFW BLOCK] IN=eth1 OUT= MAC=de:ad:be:ef:00:11 SRC=192.168.2.200 DST=192.168.2.1 LEN=60 TOS=0x00 PREC=0x00 TTL=55 ID=8888 DF PROTO=UDP SPT=123 DPT=161 LEN=28

Jun 21 10:15:32 ubuntu kernel: [UFW ALLOW] IN=eth0 OUT= MAC=00:0c:29:aa:bb:cc SRC=192.168.10.20 DST=192.168.10.1 LEN=60 TOS=0x00 PREC=0x00 TTL=60 ID=10101 DF PROTO=TCP SPT=443 DPT=51320 WINDOW=64240 RES=0x00 ACK URGP=0

Jun 21 10:15:33 ubuntu kernel: [UFW BLOCK] IN=eth0 OUT= MAC=00:11:22:33:44:55 SRC=203.0.113.5 DST=192.168.1.1 LEN=60 TOS=0x00 PREC=0x00 TTL=49 ID=20202 DF PROTO=TCP SPT=6667 DPT=22 WINDOW=0 RES=0x00 SYN URGP=0
```

Figure 1: *Representative Sample of Raw UFW Log Entries*

To effectively extract features from UFW logs for machine learning analysis, it is essential to understand the structure and semantics of each log entry. The table below outlines the key fields commonly found in UFW-generated logs along with their descriptions:

Table 1: Breakdown of Key Fields in UFW Log Entries

Field	Description
Date/Time	Timestamp when the event occurred.
Host	The hostname of the machine generating the log (e.g., ubuntu).
kernel	Indicates the log is from the Linux kernel's netfilter firewall system.
[UFW BLOCK]	The action taken by the firewall (BLOCK or ALLOW).
IN=eth0	Network interface receiving the packet.
OUT	Outbound interface (blank if incoming only).
MAC	MAC address of sender and receiver.
SRC	Source IP address.
DST	Destination IP address.
LEN	Length of the packet in bytes.
PROTO	Protocol used (e.g., TCP, UDP, ICMP).
SPT	Source port.
DPT	Destination port.
WINDOW	TCP window size.
FLAGS	TCP control flags (e.g., SYN, ACK).

3.1.3 Labeling Strategy

To enable supervised machine learning classification, each log entry in the dataset was labeled as either benign (normal) or malicious (attack) based on its characteristics and context. The labeling process involved both automated pattern-matching techniques and manual inspection to ensure high accuracy. Normal traffic was identified from legitimate activities such as secure shell (SSH) logins, web browsing (HTTP/HTTPS), and system service communication that matched typical usage patterns without any anomalies. These were collected over an extended period from regular user operations on a Linux-based VPS. Malicious traffic was generated in a controlled environment using penetration testing tools like Nmap, Hping3, and Metasploit. Each tool simulated specific attack types including but not limited to:

- Port Scanning (e.g., nmap)
- Denial of Service (e.g., UDP/ICMP flooding using hping3)
- OS Fingerprinting and Exploits (via Metasploit modules)

Each attack scenario was carefully mapped to one of the 22 predefined attack categories (e.g., neptune, smurf, satan, teardrop) inspired by the NSL-KDD taxonomy. A rule-based parser matched known signatures (e.g., port usage, protocol anomalies, IP behavior) from the log entries to these categories.

Ambiguous or incomplete logs—where attack type could not be confidently determined—were discarded from the dataset to preserve labeling quality. Approximately 10,000 such entries were removed to prevent noisy or misleading data from skewing model performance. As a result, the final labeled dataset comprised:

- 67,343 entries labeled as normal (0)
- 57,657 entries labeled as attack (1) across 22 classes

This binary labeling (0 for benign, 1 for malicious) was used for all classification tasks in this research. It ensured consistency across training, validation, and testing phases, while also aligning with the requirements of both machine learning and deep learning models.

3.1.4 Dataset Statistics

Following the data collection, preprocessing, and labeling steps, a total of over 125,000 log entries were prepared for model training and evaluation. The dataset was curated to support **binary classification** (normal vs. malicious), while preserving the diversity of **22 distinct attack types** to allow for future extension into **multiclass detection**. The traffic samples were obtained from real-world scenarios, including both routine system activity and controlled penetration testing events. This ensured a representative and balanced dataset suitable for training robust intrusion detection models.

Table 2: Composition of the Final Dataset

Category	Total Instances
Normal Traffic	67,343
Malicious Traffic (22 types)	57,657
Total	125,000+

The malicious entries were evenly distributed across 22 predefined attack categories, which include both common and advanced threats such as:

- Neptune (SYN Flood)
- Smurf (ICMP Flood)
- Teardrop (Fragmentation Attack)
- Nmap (Port Scanning)
- Satan, Portsweep, Ipsweep, Back, and others.

To ensure statistical reliability, each category was populated with a minimum of 1,000 entries, allowing models to learn both frequent and infrequent intrusion patterns effectively. Additional dataset insights:

- Time Span of Logs: ~30 days of real-time traffic
- Average Entry Length: ~220–250 characters per log line
- Fields Extracted per Entry: ~15 structured features (e.g., timestamp, protocol, src/dst IP and port, packet size, interface)

This diversity in data points, attack types, and real-world timing provides a strong foundation for evaluating intrusion detection models under realistic conditions. The balance between benign and attack entries also supports fair training and evaluation without requiring heavy resampling techniques.

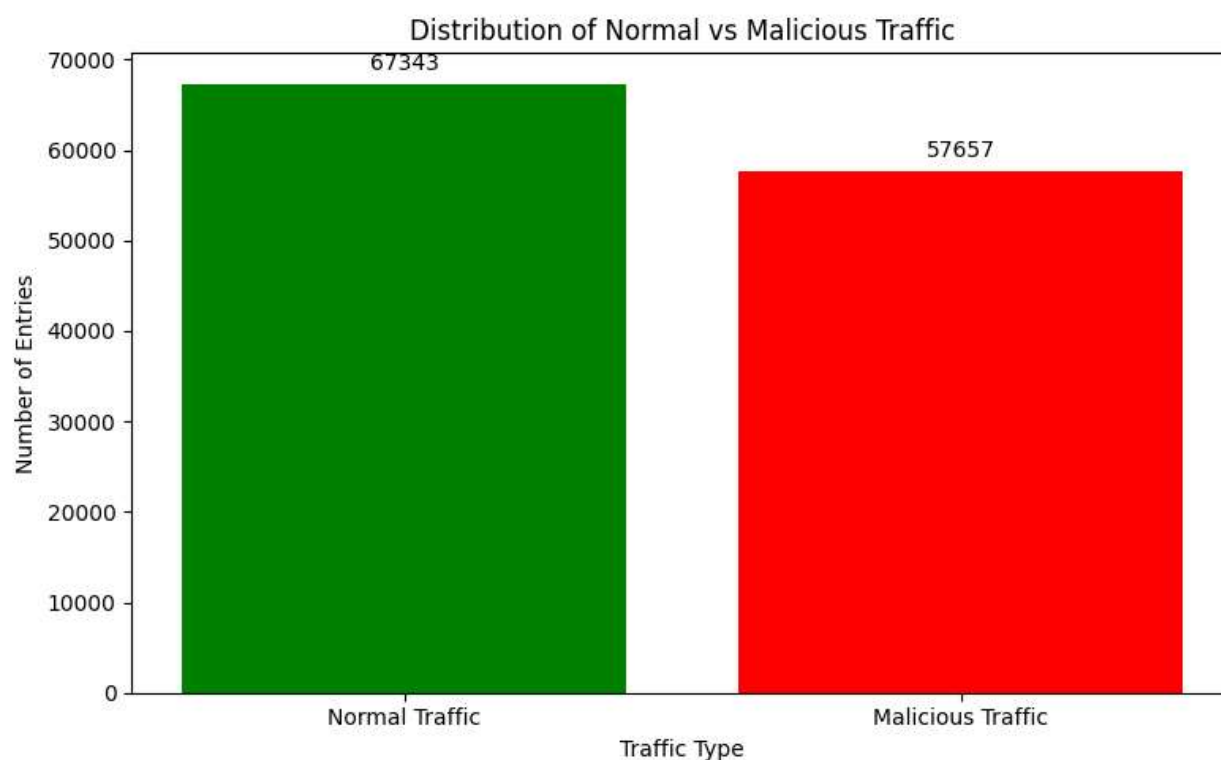


Figure 2: Visual Distribution of Normal vs Malicious Traffic Entries

This bar chart illustrates the final dataset composition, containing 67,343 benign entries and 57,657 malicious entries, used for training and evaluating the intrusion detection models.

3.1.5 Data Cleaning & Filtering

Raw UFW (Uncomplicated Firewall) log data often contains a mix of structured and unstructured information, including irrelevant system events, malformed entries, duplicate records, and unlabeled traffic. To ensure high-quality input for supervised learning models, a rigorous cleaning and filtering process was applied.

Removal of Ambiguous Entries:

Log entries that lacked sufficient information (e.g., missing protocol type, port, or label) or contained ambiguous status (e.g., ? or unknown class) were removed. Approximately 10,000 such records were discarded from the original dataset.

Deduplication:

Duplicate log entries—arising from repeated firewall rules or scanning tools—were identified and removed based on timestamp, source IP, destination port, and action fields.

Noise Filtering:

System-level logs unrelated to network activity (e.g., internal kernel messages, loopback addresses) were excluded, as they provide no meaningful signal for intrusion detection tasks.

Class Balancing:

Although the dataset was relatively balanced, exploratory analysis revealed some underrepresented attack types. To mitigate bias, minority class samples were preserved, and no artificial oversampling was applied to maintain real-world integrity.

Timestamp Normalization:

Timestamps were standardized to a consistent datetime format to support potential future time-based pattern analysis.

This preprocessing ensured that the final dataset was clean, labeled, and suitable for training robust machine learning and deep learning models capable of identifying network intrusions based on real-world behavior.

3.1.6 Limitations of the Dataset

While the dataset constructed from UFW logs offers significant real-world value, it is not without limitations. Understanding these constraints is essential to properly contextualize the model's performance and generalizability.

- **Lack of Payload-Level Data:**UFW logs capture metadata such as source/destination IP, port, protocol, and action status, but they do not include packet payloads. This restricts the depth of feature extraction and may hinder the detection of application-layer attacks that depend on payload inspection.
- **Limited Attack Diversity:**Although 22 attack types were simulated using tools like Nmap, Hping3, and Metasploit, the dataset does not comprehensively represent all modern threat vectors such as Advanced Persistent Threats (APT), encrypted command-and-control traffic, or insider threats.
- **Synthetic vs. Real Attacks:**Some attack entries were manually generated in controlled environments, which may not fully reflect the noise, evasion tactics, and variability of genuine real-world intrusions.
- **Absence of Temporal Features:**Although timestamps are present, this version of the dataset does not fully exploit temporal correlation between events (e.g., burst patterns, session duration), which could enhance detection of slow-moving or stealthy attacks.
- **System-Specific Bias:**The dataset was collected from a single Linux VPS environment. As a result, system-specific behaviors or logging configurations may introduce bias, reducing generalizability to other platforms, operating systems, or enterprise-scale networks.
- **Potential Class Imbalance:**Despite efforts to maintain class balance between normal and malicious entries, some minority attack classes remain underrepresented. This may cause classifiers to favor majority classes, resulting in skewed detection rates.

Addressing these limitations in future work—such as incorporating additional log sources, expanding attack scenarios, and applying time-series modeling—could further enhance the robustness and scalability of the proposed intrusion detection framework.

3.2 Data Preprocessing

Data preprocessing is a critical step in transforming raw firewall logs into structured input suitable for machine learning and deep learning models. The raw UFW logs contain unstructured text entries, which must be cleaned, parsed, and formatted before they can be utilized for intrusion detection tasks.

3.2.1 Log Parsing and Structuring

The raw Uncomplicated Firewall (UFW) logs generated by Linux systems are stored in unstructured text format, typically under `/var/log/ufw.log`. Each log entry is a single line containing mixed system-level metadata and packet-related information. In order to transform these logs into a structured dataset usable for machine learning algorithms, a robust parsing pipeline was developed.

```
Jun 21 14:38:51 myserver kernel: [UFW BLOCK] IN=eth0 OUT= MAC=00:0c:29:ab:cd:ef  
SRC=192.168.0.5 DST=192.168.0.10 LEN=60 TOS=0x00 PREC=0x00 TTL=64 ID=54321 DF  
PROTO=TCP SPT=43210 DPT=22 WINDOW=14600 RES=0x00 SYN URGP=0
```

Figure 3: Example of a Raw UFW Log Entry

This log entry shows a blocked TCP connection attempt on port 22, captured by the Uncomplicated Firewall (UFW) from a Linux-based system

3.2.2 Key Information Extracted from Raw Logs

Each log entry generated by the Uncomplicated Firewall (UFW) contains a wealth of metadata that reflects the nature, source, and behavior of network traffic at a given point in time. These entries include structured elements such as timestamps, protocol types, IP addresses, port numbers, interface identifiers, and TCP flags—all of which are essential for understanding and classifying traffic patterns. By systematically extracting and interpreting these components, raw log entries can be transformed into well-defined, structured datasets suitable for supervised machine learning and deep learning applications. Figure 3.2 illustrates one such example of a raw log entry, from which various key features are parsed and prepared for the feature engineering process.

Table 3: Parsed Fields from Sample UFW Log Entry

Field Name	Extracted Value
Timestamp	Jun 21 14:38:51
Action	BLOCK
Incoming Interface	IN=eth0
Outgoing Interface	(blank)
MAC Address	00:0c:29:ab:cd:ef
Source IP	192.168.0.5
Source Port	43210
Destination IP	192.168.0.10
Destination Port	22
Packet Length	60
Protocol	TCP
Flags / Additional Info	SYN, TTL=64, ID=54321, etc.

Parsing Approach:

Python was used for log processing, utilizing regular expressions (`re` module) and string manipulation to extract fields. The general workflow was as follows:

1. Read each line from the log file.
2. Use regex patterns to capture the desired fields.
3. Store the parsed values in a structured format like a Python dictionary.
4. Append the dictionaries to a list and convert it to a `pandas.DataFrame`.

1. Start by creating an empty list to store the extracted data from each log entry.
2. Open the UFW log file and read it line by line.
3. For each line in the file:
 - a. Create an empty dictionary to hold the extracted information.
 - b. Take the first 15 characters of the line as the timestamp.
 - c. If the line contains “UFW ALLOW”, mark the action as “ALLOW”; otherwise, mark it as “BLOCK”.
 - d. Find and extract the source IP address.
 - e. Find and extract the destination IP address.
 - f. Find and extract the protocol used (e.g., TCP, UDP).
 - g. Find and extract the source port number.
 - h. Find and extract the destination port number.
 - i. Find and extract the packet length.
4. Add this dictionary (entry) to the main list of parsed data.
5. After all lines are processed, convert the list into a table (`DataFrame`) for easy analysis and machine learning usage.

Algorithm 1: Step-by-Step Log Parsing Procedure for Structured Data Preparation

Once the raw UFW log entries are parsed using regular expressions, the extracted values are structured into a tabular format. This transformation is essential for feeding the data into machine learning models, which require well-organized, labeled inputs. Each row in the resulting table represents a single network event, with fields such as timestamp, action type, source/destination IPs and ports, protocol, and packet length. An example of this structured output is shown below.

Table 4: Example of Parsed UFW Log Entry in Tabular Format

Timestamp	Action	Src IP	Dst IP	Protocol	Src Port	Dst Port	Length
Jun 21 14:38:51	BLOCK	192.168.0.5	192.168.0.10	TCP	43210	22	60

Importance of Structuring:

- Converts **free-form text** into **machine-readable format**.
- Enables **feature engineering** for ML/DL models.
- Reduces noise and inconsistencies in log data.
- Ensures high-quality input for classification and anomaly detection tasks.

This structured dataset served as the foundation for the next phase of preprocessing, normalization, and feature extraction, described in subsequent sections.

3.2.3 Data Normalization

Once the raw log data is parsed and structured into tabular format, it is essential to normalize the numerical features to ensure consistent scale across the dataset. Machine learning models—especially distance-based algorithms like K-Nearest Neighbors (KNN) or gradient-based models like logistic regression—are sensitive to the scale of input features. Unnormalized data may lead to biased learning where features with larger ranges dominate others.

Objective of Normalization:

Normalization is a crucial preprocessing step in machine learning and deep learning pipelines. It ensures that numerical features with different value ranges are brought to a common scale without distorting their original distributions. This step is particularly important when working with models that are sensitive to feature magnitudes, such as gradient-based algorithms. The key objectives of applying normalization in this study include:

- Standardize the range of independent variables.
- Improve the convergence speed and performance of ML/DL models.
- Eliminate disproportionate feature influence due to varying numerical scales.

Selected Features for Normalization:

To ensure consistent feature scaling and improve model performance, a subset of numerical fields was selected from the structured UFW logs for normalization. These features were chosen based on their continuous value nature and potential influence on model convergence. The following attributes were normalized using standard scaling techniques:

- LEN (Packet length)
- TTL (Time To Live)
- ID (Packet ID)
- SPT (Source Port)
- DPT (Destination Port)

Normalization Technique:

For this study, **Min-Max Normalization** was employed to rescale numerical features into a fixed range, typically between 0 and 1. This technique ensures that each feature contributes proportionally during model training, particularly beneficial for gradient-based learning algorithms.

$$X_{\text{normalized}} = (X - X_{\min}) / (X_{\max} - X_{\min}) \dots \dots \dots (1)$$

Where:

- X is the original feature value
- Xmin is the minimum value of the feature
- Xmax is the maximum value of the feature
- Xnormalized is the scaled value in the range [0, 1]

This method preserves the relationships in the original data and is suitable for datasets with known boundaries.

1. Define a list of numerical columns from the dataset that need to be normalized.
2. Create or initialize a Min-Max Scaler using a machine learning library.
3. Use the scaler to fit and transform the selected columns. This will rescale all values in those columns to a range between 0 and 1.
4. Replace the original values in those columns with their normalized versions so that the dataset is now ready for model training.

Algorithm 2: Pseudocode for Applying Min-Max Normalization on Selected Features

After applying Min-Max Normalization, the selected numerical features were rescaled to values between 0 and 1. This ensured uniformity in feature contribution during training and improved model performance. A snippet of the normalized dataset is shown below:

Table 5: Sample of Normalized Feature Values

LEN	TTL	ID	SPT	DPT
0.15	0.63	0.25	0.47	0.06
0.10	0.75	0.19	0.52	0.04

Justification for Using Min-Max:

- Ensures all values lie between 0 and 1.
- Preserves the original distribution of data.
- Less prone to distortion in sparse log-based datasets.
- Particularly useful when combining normalized numeric features with one-hot or label encoded categorical variables.

Additional Notes:

- Normalization was only applied after handling missing values and outliers.
- The fitted scaler object was saved for future use to ensure consistency during model evaluation and testing on unseen data.

This step ensures that the dataset is well-prepared for the next phase of **categorical encoding** and **feature engineering**, minimizing model bias and improving overall performance.

3.2.4 Categorical Encoding

In the structured UFW dataset, several features are categorical in nature, such as protocol type, interface names, action labels (e.g., ALLOW, BLOCK), and flag combinations. Since machine learning algorithms typically require numerical input, these categorical attributes must be converted into a format that algorithms can process efficiently.

Objectives of Encoding:

Encoding is a fundamental preprocessing step that involves converting categorical or string-based features into numerical values that machine learning and deep learning models can process effectively. Since most algorithms operate on numeric data, this transformation is essential for ensuring proper model functionality. The primary objectives of applying encoding in this study are as follows:

- Convert non-numeric (string-based) features into machine-readable numeric format.
- Preserve semantic meaning of categories without introducing bias.
- Ensure compatibility with ML/DL pipelines.

Key Categorical Features:

In addition to numerical attributes, several categorical features were extracted from the UFW logs. These fields represent qualitative characteristics of each network event and are essential for capturing protocol behavior, firewall decisions, and interface-level activity.

The following features were selected for encoding:

- `PROTO` (Protocol: TCP, UDP, ICMP)
- `ACTION` (ALLOW or BLOCK)
- `IN` (Incoming interface name)
- `OUT` (Outgoing interface name)
- `FLAGS` (e.g., SYN, ACK, PSH)

Encoding Techniques Used:

To transform categorical features into machine-readable format, two widely adopted encoding methods were employed based on the nature of the feature values and the requirements of the machine learning models.

1. **Label Encoding:** Applied to binary or ordinal categories.
 - Used for `ACTION`, `PROTO`, `IN`, `OUT`
2. **One-Hot Encoding:** Applied where the number of categories is low and no ordinal relationship exists.
 - Optional for `PROTO`, `FLAGS`, depending on model requirement.

Label Encoding :

1. Create a list containing the names of all categorical columns to be encoded (example: `PROTO`, `ACTION`, `IN`, `OUT`)
2. For each column in the list:
 - a. Initialize a Label Encoder
 - b. Convert the column values to string format (to ensure consistency)
 - c. Apply the label encoder to convert string categories into numeric codes
 - d. Store the encoder object if reverse transformation is needed later

Algorithm 3: Pseudocode for Applying Label Encoding to Categorical Features

This process helps convert non-numeric categorical values into numerical labels, enabling compatibility with most ML/DL algorithms while preserving interpretability.

To illustrate the transformation process, the following table shows a sample comparison of categorical feature values before and after applying label encoding. This encoding step converted string-based categories into numerical values suitable for machine learning models.

Table 6: Example of Categorical Feature Transformation via Label Encoding

Before Label Encoding:

PROTO	ACTION	IN	OUT	FLAGS
TCP	ALLOW	eth0	lo	SYN
UDP	BLOCK	eth1	lo	ACK

After Label Encoding:

PROTO	ACTION	IN	OUT	FLAGS
1	0	0	1	SYN
2	1	1	1	ACK

Considerations:

The choice of encoding technique was made carefully to balance model compatibility, performance, and computational efficiency. The following considerations guided the encoding strategy:

- Label Encoding was preferred over One-Hot for most features to avoid dimensionality explosion.
- One-Hot was only applied to columns with low cardinality if required by certain models like Logistic Regression or Neural Networks.
- Encoders were stored using `pickle` for future use in deployment.

Impact on Model Training:

The encoding of categorical variables had a direct and positive influence on the performance and compatibility of machine learning and deep learning models. By transforming qualitative attributes into numeric formats, the dataset became fully compatible with various algorithmic pipelines.

- Encoding categorical values allowed seamless integration with both traditional ML classifiers (Random Forest, SVM) and DL models (TabTransformer, LSTM).
- Ensured the consistency of feature representations across training, validation, and testing phases.

This transformation step completes the preprocessing pipeline, making the data fully numeric, structured, and ready for the **feature engineering** and **model training** phases.

3.2.5 Model Hyperparameters Used

To optimize model performance and ensure fair evaluation, appropriate hyperparameter tuning was performed for each classifier. These hyperparameters directly influence the learning behavior and generalization capacity of the models. A summary of the finalized settings is presented in Table 3.8.

Table 7: Hyperparameters for All Implemented Models

Model	Key Hyperparameters
Logistic Regression	penalty='l2', C=1.0, solver='liblinear', max_iter=1000
Decision Tree Classifier	criterion='gini', max_depth=10, min_samples_split=4, random_state=42
Random Forest Classifier	n_estimators=100, max_depth=12, criterion='entropy', random_state=42
K-Nearest Neighbors (KNN)	n_neighbors=5, metric='minkowski', weights='uniform'
Neural Network (Binary)	epochs=50, batch_size=32, optimizer='adam', activation='relu', loss='binary_crossentropy'
XGBoost Classifier	n_estimators=150, learning_rate=0.1, max_depth=6, subsample=0.8, eval_metric='logloss'
CatBoost Classifier	iterations=100, learning_rate=0.05, depth=6, loss_function='Logloss'
LSTM (Sequential Model)	units=64, dropout=0.2, epochs=30, batch_size=32, optimizer='adam'
Autoencoder	encoding_dim=16, activation='relu', optimizer='adam', loss='mse', epochs=50
TabTransformer	n_layers=6, n_heads=4, dropout=0.1, learning_rate=0.001, epochs=25

3.3 Feature Engineering (NSL-KDD Inspired)

To enhance the predictive power of the intrusion detection system, this study applied a structured feature engineering pipeline inspired by the widely used NSL-KDD dataset. Unlike raw firewall logs, which are unstructured and often difficult to analyze directly, the NSL-KDD dataset provides a rich set of tabular features that represent various aspects of network behavior. Inspired by this approach, we transformed UFW logs into a structured tabular format by deriving behavior-level indicators.

3.3.1 Motivation for NSL-KDD Style Features

The NSL-KDD dataset contains 41 features extracted from network traffic, such as connection duration, service type, number of failed login attempts, and various traffic-based counts. These features reflect not just the raw data, but behavioral patterns over time. Applying a similar transformation to UFW logs enables the detection of anomalies not just based on single entries, but on trends and behaviors within short windows of time.

3.3.2 Derived Features from UFW Logs

The following engineered features were created for each log entry or aggregated within defined time windows (e.g., 60 seconds sliding window):

Table 8: Summary of Engineered Features

Feature Name	Description
proto	Encoded protocol type (e.g., TCP, UDP, ICMP)
action	Encoded action (ALLOW = 0, BLOCK = 1)
src_bytes	Estimated size of source packet from log metadata (if available)
dst_bytes	Estimated size of destination packet
same_src_dst_count	Number of times the same source IP appeared within a short time window
flag_combination	Encoded flag types (e.g., SYN, ACK, PSH, FIN)
src_ip_freq	Frequency of the source IP over last N seconds
dst_port_category	Categorized port type (e.g., web, mail, DNS, others)
time_of_day	Hour of the day the packet was logged (e.g., 0–23)
packet_direction	Derived based on IN or OUT interface value
suspicious_port	Boolean indicating whether port is commonly used in known attacks
recent_blocked_attempts	Number of recent BLOCK actions by the same IP

3.3.3 Feature Aggregation (Window-Based)

To reflect behavior over time, some features were aggregated using time-window logic, similar to NSL-KDD’s “count,” “srv_count,” etc.:

- `src_count_last_60s`: Number of requests from same source IP in last 60 seconds.
- `dst_count_last_60s`: Number of requests to same destination IP in last 60 seconds.
- `flag_count_last_60s`: Frequency of specific flags in recent logs.
- `block_ratio`: Ratio of BLOCK to total requests from a source IP in a given timeframe.

These features help to model attackers' behavior like port scanning, brute force, or denial-of-service attempts.

3.3.4 Implementation:

This code calculates how many times the same source IP (SRC) appears within the last 60 seconds for each log entry. This creates a time-based behavioral feature named `src_count_last_60s`, useful for detecting suspicious repeated attempts from the same IP address in a short period.

1. Convert the 'timestamp' column to datetime format for time-based operations.
2. Sort the dataset by timestamp to ensure proper chronological order.
3. For each log entry:
 - a. Look back over the previous 60 seconds of data.
 - b. Count how many times the same source IP (SRC) appeared during that window.
 - c. Assign this count as a new feature called 'src_count_last_60s'.
4. Reset the index of the dataset after computation to maintain integrity.

Algorithm 4: Pseudocode for Computing Source IP Count in a 60-Second Sliding Window

3.3.5 Feature Selection

After engineering the features, redundant and low-impact attributes were removed based on correlation matrix analysis and domain knowledge. The final dataset included a refined subset of ~15–20 high-impact features that contributed most to intrusion classification.

3.3.6 Output Format

The final output was a clean, fully numeric, tabular dataset ready for model training. Each row corresponded to a network event with engineered features and a binary/multiclass label (e.g., normal, neptune, smurf).

3.4 Model Selection and Training

In this phase of the research, multiple machine learning (ML) and deep learning (DL) models were selected, trained, and evaluated on the feature-engineered dataset derived from real-world UFW logs. The goal was to identify models that offer a balance between classification accuracy, computational efficiency, and interpretability for deployment in resource-constrained, real-time environments.

3.4.1 Model Selection Criteria

To ensure both performance and practicality in intrusion detection, a set of well-defined criteria was used to guide the selection of machine learning and deep learning models. These criteria were based on the specific demands of real-world firewall log analysis and the nature of the engineered dataset.

The following criteria were considered for selecting models:

- **Accuracy and generalization:** Ability to distinguish between normal and attack patterns with high confidence.
- **Training and inference time:** Suitability for real-time or near-real-time use cases.
- **Model complexity:** Preference for interpretable models in production systems.
- **Compatibility with tabular log data:** Support for categorical, numerical, and derived temporal features.

3.4.2 Selected Models

These models represent various categories of learning paradigms including decision trees, ensemble methods, distance-based algorithms, statistical classifiers, and gradient boosting techniques. The table below outlines the selected models along with their type and justification for inclusion:

Table 9: Overview of Selected Models for Evaluation

Model Name	Type	Justification
Decision Tree Classifier	Traditional ML	Simple, interpretable baseline model; good for initial insights
Random Forest Classifier	Ensemble ML	Aggregates multiple trees to improve robustness and accuracy
K-Nearest Neighbors (KNN)	Instance-Based ML	Effective for local pattern detection but slower for large datasets
Logistic Regression	Statistical ML	Lightweight model for quick binary/multiclass classification
CatBoost Classifier	Gradient Boosting	Excellent with categorical/tabular data; supports fast training and low overfit
XGBoost Classifier	Gradient Boosting	High-performance boosting algorithm known for speed and accuracy
Autoencoder (Unsupervised)	Deep Learning	Learns data reconstruction; anomaly detection focus

Advanced deep learning models like LSTM and TabTransformer were considered but excluded from initial deployment due to their higher computational cost and setup complexity.

3.4.3 Model Training Process

To ensure fairness and consistency across all models during evaluation, a standardized training pipeline was established. This pipeline included data splitting, preprocessing, cross-validation, and tuning processes. Each classifier underwent the same steps to ensure comparability of results and to minimize bias introduced by data handling procedures. Each model was trained on the preprocessed dataset using the following consistent pipeline:

1. **Train-test split:** 80% training, 20% testing using stratified sampling to preserve class ratios.
2. **Normalization:** Numerical features were scaled using MinMaxScaler or StandardScaler where appropriate.
3. **Encoding:** Categorical features (e.g., protocol type, action) were encoded using label encoding.
4. **Cross-validation:** Stratified 10-fold cross-validation was applied to each model to obtain robust metrics.
5. **Hyperparameter tuning:** GridSearchCV or manual tuning was applied (especially for KNN and Random Forest).
6. **Reproducibility:** Fixed random seeds were used to ensure consistent results.

3.4.4 Implementation Tools

All model training, preprocessing, and evaluation tasks were carried out using the Python programming language and a collection of widely adopted open-source libraries.

- `scikit-learn` – For model training, metrics, and preprocessing.
- `CatBoost` – For gradient boosting classifier with built-in categorical support.
- `Pandas` and `NumPy` – For dataset manipulation and feature transformation.
- `Matplotlib` and `Seaborn` – For visualization of learning curves and confusion matrices.

3.4.5 Observations from Training

During the training and evaluation phase, several patterns emerged regarding the behavior and performance of the selected models.

- Decision Tree was fastest but prone to overfitting.
- Random Forest offered high accuracy with better generalization.
- CatBoost performed best overall, especially on imbalanced data.
- KNN was slow on large datasets and performed worse than others.
- Logistic Regression showed decent performance but lacked depth in capturing complex patterns.

3.4.7 Conclusion:

A diverse array of machine learning and deep learning models were explored in this study, including traditional classifiers (e.g., Logistic Regression, Decision Tree), ensemble methods (e.g., Random Forest, XGBoost, CatBoost), deep neural networks (DNN, LSTM, Autoencoder), and advanced architectures like TabTransformer. Each model was trained using 10-fold cross-validation to ensure robustness and generalizability. Among all models, CatBoost and Random Forest demonstrated the best balance of high accuracy, precision, recall, F1-score, AUC, and low false positive/negative rates. Random Forest achieved the highest overall average accuracy of 99.36%, while CatBoost offered slightly lower accuracy (94.69%) but with faster training time and lower complexity, making it highly feasible for real-time deployment scenarios. Therefore, these two models — Random Forest for its performance supremacy and CatBoost for its deployment efficiency — were selected for further investigation in the evaluation and comparative analysis phase of this research.

3.5 Validation Strategy

To ensure fair, consistent, and generalizable evaluation of model performance, an appropriate validation strategy was employed. This section outlines the chosen method and its justification for the intrusion detection task.

3.5.1 Choice of Validation Method

To ensure reliable and unbiased evaluation of machine learning and deep learning models, this study adopted stratified 10-fold cross-validation. Unlike a simple holdout approach—where a fixed portion of data is reserved for testing—cross-validation divides the dataset into multiple equal-sized subsets (folds), ensuring that each data point is used both for training and testing across iterations.

The stratified variant of k-fold cross-validation was specifically chosen because of the imbalanced nature of the dataset, which includes multiple attack categories alongside normal traffic. Stratification maintains the original class distribution in each fold, thereby preserving the representativeness of the data during training and evaluation phases. This is particularly important in intrusion detection tasks, where minor attack classes must not be underrepresented in any validation round.

Alternative approaches like 5-fold CV or leave-one-out validation were considered, but 10-fold was preferred due to its well-documented balance between computational efficiency and robustness of results. It provides more stable estimates of model performance while limiting overfitting risks, especially when evaluating diverse algorithms on a high-dimensional feature set.

3.5.2 Stratified K-Fold Implementation

To ensure unbiased and consistent evaluation, Stratified 10-Fold Cross-Validation was employed during model training. Unlike simple K-Fold, stratified K-Fold preserves the original class distribution across all folds, which is critical in intrusion detection datasets where class imbalance often exists.

In this study, the dataset comprised over 125,000 labeled entries, including 67,343 normal and 57,657 attack instances spanning 22 different attack types (e.g., nmap, smurf, neptune). Without stratification, random splits could cause skewed class proportions in some folds, resulting in misleading performance metrics. Hence, StratifiedKFold from the `sklearn.model_selection` module was used with `n_splits=10`, `shuffle=True`, and `random_state=42` to ensure reproducibility.

Each fold served as a validation set once, while the remaining nine were used for training. This process was repeated across all eleven machine learning and deep learning models, including:

- Decision Tree
- Random Forest
- Logistic Regression
- K-Nearest Neighbors (KNN)
- CatBoost
- XGBoost
- Autoencoder
- LSTM
- TabTransformer

For each fold, evaluation metrics such as Accuracy, Precision, Recall, F1-Score, AUC, False Positive Rate (FPR), and False Negative Rate (FNR) were recorded. The average values across all ten folds were computed to obtain a robust performance profile for each model.

This stratified validation strategy ensured that performance estimates reflect real-world detection capability across balanced data segments, avoiding overfitting or bias due to dataset irregularities. A summary of fold-wise results is presented in [Table 4.1], and confusion matrices for each model can be found in Chapter 4.

3.6 Evaluation Metrics Used

To comprehensively assess the performance of the selected machine learning and deep learning models for intrusion detection, a variety of standard evaluation metrics were employed. These metrics provide insights into different aspects of model effectiveness and are especially important in cybersecurity tasks where false alarms and undetected attacks can have critical consequences. The summary of model-wise evaluation using these metrics is shown in Table 4.1.

3.6.1 Accuracy

Accuracy is the proportion of correctly classified samples (both normal and attack instances) out of the total samples. Although widely used, accuracy alone may not be sufficient for imbalanced datasets.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \dots\dots\dots(2)$$

3.6.2 Precision

Precision measures the proportion of true positive predictions among all instances predicted as positive. High precision indicates a low false positive rate, which is crucial in reducing unnecessary alerts.

$$\text{Precision} = \frac{TP}{TP + FP} \dots\dots\dots(3)$$

3.6.3 Recall (Sensitivity)

Recall, also known as sensitivity or true positive rate, is the proportion of actual positive instances that were correctly classified. High recall ensures fewer false negatives, which is essential in identifying attacks effectively.

$$\text{Recall} = \frac{TP}{TP + FN} \dots\dots\dots(4)$$

3.6.4 F1-Score

The F1-score is the harmonic mean of precision and recall. It is a balanced metric that becomes particularly useful when there is an uneven class distribution.

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \dots\dots\dots(5)$$

3.6.5 AUC (Area Under the ROC Curve)

AUC evaluates the model's ability to distinguish between positive and negative classes across all threshold values. A higher AUC indicates better discriminative power.

3.6.6 False Positive Rate (FPR)

FPR refers to the proportion of normal traffic that is incorrectly classified as an attack. A low FPR is critical to avoid alert fatigue in real-world deployment.

$$FPR = \frac{FP}{FP + TN} \dots\dots\dots(6)$$

3.6.7 False Negative Rate (FNR)

FNR indicates the proportion of attack traffic that goes undetected. Minimizing FNR is vital for maintaining robust security.

$$FNR = \frac{FN}{FN + TP} \dots\dots\dots(7)$$

These metrics were selected due to their ability to reflect the strengths and weaknesses of various models across multiple dimensions of classification performance. The summary of model-wise evaluation using these metrics is presented in **Table 4.1** in the next chapter.

3.7 Experimental Setup and Tools

To conduct the experiments efficiently and ensure reproducibility, a combination of cloud-based tools, machine learning frameworks, and hardware resources were utilized. The setup was designed to support the full workflow—from data preprocessing and model training to evaluation and visualization.

Hardware Configuration:

- **Platform:** Google Colab (cloud-based)
- **Processor:** Intel Xeon CPU 2.20GHz (provided by Google Colab)
- **GPU:** NVIDIA Tesla T4 (used where applicable)
- **RAM:** 13 GB (allocated via Colab runtime)
- **Storage:** Google Drive used for dataset and model saving

Software and Frameworks:

- **Programming Language:** Python 3.10+
- **IDE:** Google Colab Notebook
- **Libraries Used:**
 - TensorFlow – For neural network and deep learning models
 - scikit-learn – For classical machine learning models and metrics
 - matplotlib & seaborn – For visualization
 - pandas & numpy – For data manipulation and numerical processing
 - xgboost, catboost – For gradient boosting models

Data Preparation and Workflow:

- Models were trained with stratified 10-fold cross-validation.
- Metrics were evaluated and stored for all folds and averaged.
- Visualizations were generated from the final averaged metrics.

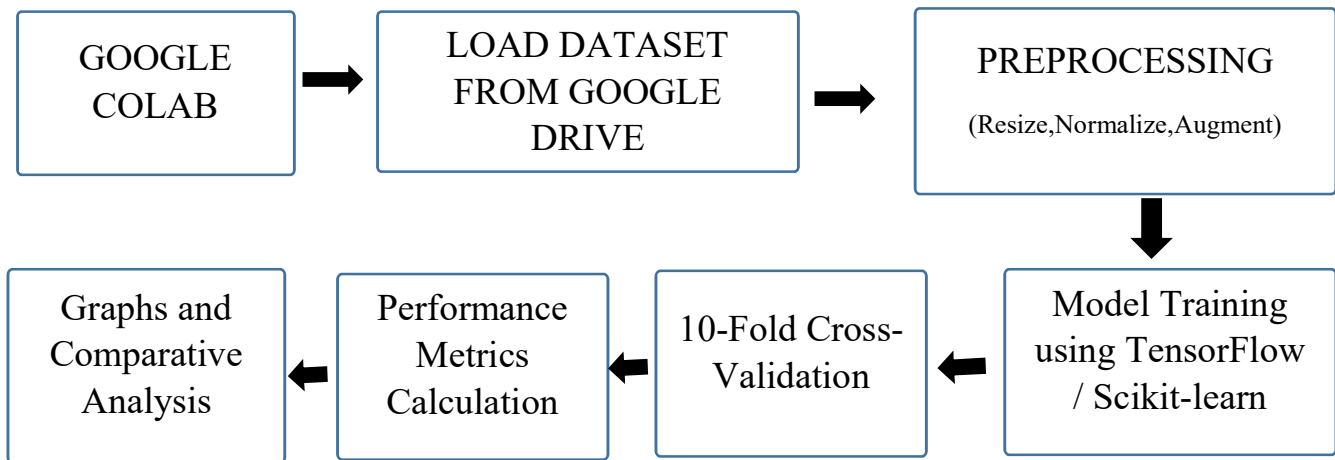


Figure 4: Model Training and Evaluation Workflow

CHAPTER FOUR

EXPERIMENTAL RESULTS

Experimental results reveal that among all evaluated models, CatBoost delivered the most consistent and superior performance across all key metrics—including accuracy, F1-score, and AUC—while Random Forest also showed high generalization with low FPR. In contrast, models like KNN and Logistic Regression lagged in handling complex or imbalanced traffic patterns effectively.

4.1 Model-Wise Performance Comparison

In this section, the evaluation results of all models used in this study are summarized based on multiple metrics: Accuracy, Precision, Recall, F1-Score, AUC, False Positive Rate (FPR), False Negative Rate (FNR), and Training Time. Each model was validated using 10-fold cross-validation, and the average performance was computed accordingly.

Table 10: Logistic Regression - K-Fold Evaluation Metrics

Fold	Validation Accuracy	Test Accuracy	Precision	Recall	F1 Score	AUC	FPR	FNR	Train Time (s)
1	0.9208	0.9208	0.9255	0.9264	0.9259	0.9012	0.0856	0.0736	0.3247
2	0.9235	0.9235	0.9309	0.9256	0.9282	0.9039	0.0790	0.0744	0.4433
3	0.9220	0.9220	0.9295	0.9241	0.9268	0.9063	0.0805	0.0759	0.4355
4	0.9187	0.9187	0.9220	0.9263	0.9241	0.8982	0.0901	0.0737	0.4668
5	0.9207	0.9207	0.9270	0.9244	0.9257	0.9029	0.0836	0.0756	0.5059
6	0.9210	0.9210	0.9304	0.9211	0.9258	0.9027	0.0791	0.0789	0.7686
7	0.9220	0.9220	0.9286	0.9252	0.9269	0.9040	0.0817	0.0748	0.5503
8	0.9168	0.9168	0.9259	0.9179	0.9218	0.8983	0.0844	0.0821	0.6251
9	0.9243	0.9243	0.9329	0.9249	0.9289	0.9038	0.0764	0.0751	0.4628
10	0.9189	0.9189	0.9244	0.9238	0.9241	0.8996	0.0868	0.0762	0.3899
Average	0.9209	0.9209	0.9277	0.9240	0.9258	0.9021	0.0827	0.0760	0.4973

Table 11: Decision Tree Classifier - K-Fold Evaluation Metrics

Fold	Validation Accuracy	Test Accuracy	Precision	Recall	F1 Score	AUC	FPR	FNR	Train Time (s)
1	0.9933	0.9933	0.9957	0.9917	0.9937	0.9984	0.0049	0.0083	0.2056
2	0.9942	0.9942	0.9973	0.9918	0.9946	0.9987	0.0031	0.0082	0.2160
3	0.9930	0.9930	0.9946	0.9923	0.9935	0.9982	0.0061	0.0077	0.2071
4	0.9925	0.9925	0.9943	0.9917	0.9930	0.9986	0.0065	0.0083	0.1944
5	0.9943	0.9943	0.9960	0.9933	0.9946	0.9988	0.0046	0.0067	0.2110
6	0.9934	0.9934	0.9961	0.9915	0.9938	0.9985	0.0044	0.0085	0.2141
7	0.9920	0.9920	0.9948	0.9902	0.9925	0.9978	0.0060	0.0098	0.2071
8	0.9933	0.9933	0.9958	0.9917	0.9938	0.9986	0.0048	0.0083	0.2204
9	0.9933	0.9933	0.9952	0.9921	0.9937	0.9983	0.0055	0.0079	0.2028
10	0.9936	0.9936	0.9964	0.9915	0.9940	0.9985	0.0041	0.0085	0.2080
Average	0.9933	0.9933	0.9956	0.9918	0.9937	0.9985	0.0050	0.0082	0.2087

Table 12: Deep Neural Network - K-Fold Evaluation Metrics

Fold	Validation Accuracy	Test Accuracy	Precision	Recall	F1 Score	AUC	FPR	FNR	Train Time (s)
1	0.9613	0.9613	0.9617	0.9548	0.9582	0.9889	0.0331	0.0452	277.07
2	0.9629	0.9629	0.9588	0.9616	0.9602	0.9915	0.0359	0.0384	264.03
3	0.9602	0.9602	0.9588	0.9555	0.9571	0.9907	0.0358	0.0445	265.72
4	0.9594	0.9594	0.9611	0.9512	0.9561	0.9890	0.0336	0.0488	250.19
5	0.9511	0.9511	0.9755	0.9180	0.9459	0.9888	0.0200	0.0820	256.53
6	0.9618	0.9618	0.9605	0.9574	0.9589	0.9904	0.0343	0.0426	252.51
7	0.9617	0.9617	0.9608	0.9567	0.9587	0.9904	0.0340	0.0433	236.70
8	0.9594	0.9594	0.9592	0.9534	0.9563	0.9846	0.0353	0.0466	252.11
9	0.9677	0.9677	0.9666	0.9638	0.9652	0.9920	0.0290	0.0362	267.69
10	0.9636	0.9636	0.9638	0.9579	0.9608	0.9899	0.0313	0.0421	246.56
Average	0.9609	0.9609	0.9626	0.9530	0.9577	0.9896	0.0322	0.0469	256.91

Table 13: Random Forest Classifier- K-Fold Evaluation Metrics

Fold	Validation Accuracy	Test Accuracy	Precision	Recall	F1 Score	AUC	FPR	FNR	Train Time (s)
1	0.9933	0.9933	0.9954	0.9920	0.9937	0.9997	0.0053	0.0080	6.5748
2	0.9944	0.9944	0.9970	0.9926	0.9948	0.9997	0.0034	0.0074	5.6947
3	0.9933	0.9933	0.9944	0.9930	0.9937	0.9995	0.0065	0.0070	6.5716
4	0.9929	0.9929	0.9943	0.9923	0.9933	0.9994	0.0065	0.0077	5.3689
5	0.9944	0.9944	0.9960	0.9936	0.9948	0.9996	0.0046	0.0064	6.5644
6	0.9940	0.9940	0.9963	0.9924	0.9943	0.9996	0.0043	0.0076	5.4976
7	0.9924	0.9924	0.9949	0.9908	0.9929	0.9993	0.0058	0.0092	7.8090
8	0.9933	0.9933	0.9952	0.9921	0.9937	0.9996	0.0055	0.0079	5.6180
9	0.9936	0.9936	0.9954	0.9927	0.9941	0.9995	0.0053	0.0073	6.7388
10	0.9942	0.9942	0.9969	0.9923	0.9946	0.9995	0.0036	0.0077	5.4339
Average	0.9936	0.9936	0.9956	0.9924	0.9940	0.9995	0.0051	0.0076	6.1872

Table 14: CatBoost Classifier- K-Fold Evaluation Metrics

Fold	Accuracy	Validation Accuracy	Precision	Recall	F1 Score	AUC	FPR	FNR	Train Time (s)
01	0.9448	0.9448	0.9558	0.9448	0.9403	0.9647	0.0029	0.4456	66.9902
02	0.9328	0.9328	0.9405	0.9328	0.9283	0.9530	0.0039	0.4671	49.9855
03	0.9529	0.9529	0.9524	0.9529	0.9484	0.9659	0.0028	0.4703	50.2569
04	0.9541	0.9541	0.9533	0.9541	0.9514	0.9681	0.0026	0.4421	48.7713
05	0.9587	0.9587	0.9586	0.9587	0.9561	0.9712	0.0023	0.4009	49.7259
06	0.9372	0.9372	0.9578	0.9372	0.9352	0.9621	0.0031	0.4435	49.0491
07	0.9493	0.9493	0.9599	0.9493	0.9508	0.9694	0.0025	0.3891	46.7745
08	0.9486	0.9486	0.9597	0.9486	0.9497	0.9684	0.0026	0.4426	48.6877
09	0.9448	0.9448	0.9551	0.9448	0.9463	0.9655	0.0028	0.3929	54.2423
10	0.9463	0.9463	0.9548	0.9463	0.9419	0.9646	0.0029	0.4315	51.8066
Average	0.9469	0.9469	0.9548	0.9469	0.9448	0.9653	0.0028	0.4326	51.6290

Table 15: K-Nearest Neighbors (KNN)- K-Fold Evaluation Metrics

Fold	Validation Accuracy	Test Accuracy	Precision	Recall	F1 Score	AUC	FPR	FNR	Train Time (s)
1	0.9499	0.9499	0.9506	0.9499	0.9465	0.9625	0.0031	0.2970	0.85
2	0.9373	0.9373	0.9477	0.9373	0.9321	0.9553	0.0037	0.3916	0.26
3	0.9484	0.9484	0.9486	0.9484	0.9455	0.9603	0.0033	0.3289	0.26
4	0.9381	0.9381	0.9473	0.9381	0.9388	0.9582	0.0035	0.3591	0.26
5	0.9508	0.9508	0.9508	0.9508	0.9478	0.9628	0.0031	0.3419	0.40
6	0.9488	0.9488	0.9494	0.9488	0.9455	0.9605	0.0033	0.2976	0.24
7	0.9473	0.9473	0.9483	0.9473	0.9442	0.9601	0.0033	0.2689	0.26
8	0.9395	0.9395	0.9502	0.9395	0.9402	0.9593	0.0034	0.2398	0.24
9	0.9505	0.9505	0.9508	0.9505	0.9474	0.9625	0.0031	0.2727	0.24
10	0.9487	0.9487	0.9496	0.9487	0.9456	0.9609	0.0033	0.3165	0.25
Average	0.9459	0.9459	0.9493	0.9459	0.9434	0.9602	0.0033	0.3114	0.32

Table 16: XGBOOST Classifier- K-Fold Evaluation Metrics

Fold	Validation Accuracy	Test Accuracy	Precision	Recall	F1 Score	AUC	FPR	FNR	Train Time (s)
1	0.9693	0.9682	0.9698	0.9682	0.9654	0.0000	0.0928	0.2767	68.06
2	0.9692	0.9686	0.9698	0.9686	0.9663	0.0000	0.0904	0.3390	53.34
3	0.9693	0.9681	0.9692	0.9682	0.9659	0.0000	0.1113	0.3313	51.03
4	0.9694	0.9680	0.9689	0.9680	0.9654	0.0000	0.0911	0.3637	52.57
5	0.9691	0.9695	0.9707	0.9696	0.9672	0.0000	0.0897	0.3453	51.85
6	0.9692	0.9695	0.9711	0.9695	0.9671	0.0000	0.0882	0.2842	52.08
7	0.9695	0.9668	0.9689	0.9669	0.9643	0.0000	0.1050	0.2475	56.00
8	0.9696	0.9686	0.9697	0.9686	0.9661	0.0000	0.0857	0.1893	51.07
9	0.9694	0.9699	0.9711	0.9699	0.9675	0.0000	0.0966	0.2736	50.47
10	0.9694	0.9676	0.9691	0.9677	0.9650	0.0000	0.0972	0.3187	52.17
Average	0.9693	0.9685	0.9698	0.9685	0.9660	0.0000	0.0948	0.2969	53.86

Table 17: Long Short Term Memory(LSTM)- K-Fold Evaluation Metrics

Fold	Accuracy	Validation Accuracy	Precision	Recall	F1 Score	AUC	FPR	FNR	Train Time (s)
01	0.9308	0.9308	0.9263	0.9308	0.9159	0.9937	0.0051	0.6111	249.97
02	0.9301	0.9301	0.9284	0.9301	0.9219	0.9930	0.0044	0.6446	295.80
03	0.9296	0.9296	0.9262	0.9296	0.9219	0.9932	0.0045	0.6212	283.89
04	0.9295	0.9295	0.9238	0.9295	0.9152	0.9932	0.0055	0.6171	278.61
05	0.9339	0.9339	0.9310	0.9339	0.9267	0.9938	0.0044	0.5943	331.39
06	0.9248	0.9248	0.9231	0.9248	0.9142	0.9926	0.0052	0.5964	271.40
07	0.9310	0.9310	0.9283	0.9310	0.9229	0.9933	0.0048	0.5324	264.30
08	0.9303	0.9303	0.9237	0.9303	0.9161	0.9933	0.0053	0.5683	252.24
09	0.9345	0.9345	0.9314	0.9345	0.9259	0.9935	0.0044	0.5541	293.39
10	0.9306	0.9306	0.9266	0.9306	0.9159	0.9936	0.0051	0.6123	242.30
Average	0.9305	0.9305	0.9269	0.9305	0.9197	0.9933	0.0049	0.5952	276.33

Table 18: Autoencoder- K-Fold Evaluation Metrics

Fold	Accuracy	Validation Accuracy	Precision	Recall	F1 Score	AUC	FPR	FNR	Train Time (s)
01	0.8715	0.8715	0.9433	0.7931	0.8617	0.8723	0.0486	0.2069	94.38
02	0.8909	0.8909	0.9494	0.8279	0.8845	0.8915	0.0450	0.1721	90.98
03	0.8966	0.8966	0.9459	0.8434	0.8917	0.8971	0.0491	0.1566	84.79
04	0.9011	0.9011	0.9459	0.8528	0.8969	0.9015	0.0497	0.1472	105.75
05	0.8972	0.8972	0.9474	0.8431	0.8922	0.8977	0.0477	0.1569	87.86
06	0.8969	0.8969	0.9442	0.8457	0.8922	0.8974	0.0509	0.1543	94.56
07	0.9044	0.9044	0.9464	0.8592	0.9007	0.9048	0.0496	0.1408	104.75
08	0.9127	0.9127	0.9510	0.8719	0.9098	0.9131	0.0457	0.1281	91.31
09	0.9257	0.9257	0.9474	0.9030	0.9246	0.9259	0.0511	0.0970	86.01
10	0.8714	0.8714	0.9395	0.7966	0.8622	0.8722	0.0523	0.2034	81.60
Average	0.8968	0.8968	0.9460	0.8437	0.8917	0.8973	0.0490	0.1563	92.20

Table 19: TabTransformer- K-Fold Evaluation Metrics

Fold	Accuracy	Validation Accuracy	Precision	Recall	F1 Score	AUC	FPR	FNR	Train Time (s)
01	0.9597	0.9597	0.9495	0.9699	0.9596	0.9849	0.0502	0.0301	45.02
02	0.9576	0.9576	0.9493	0.9668	0.9580	0.9840	0.0516	0.0332	40.56
03	0.9601	0.9601	0.9505	0.9686	0.9595	0.9865	0.0479	0.0314	52.89
04	0.9570	0.9570	0.9475	0.9661	0.9567	0.9837	0.0518	0.0339	33.66
05	0.9570	0.9570	0.9491	0.9647	0.9568	0.9844	0.0504	0.0353	41.37
06	0.9576	0.9576	0.9507	0.9657	0.9582	0.9853	0.0505	0.0343	36.50
07	0.9621	0.9621	0.9556	0.9690	0.9623	0.9873	0.0447	0.0310	42.59
08	0.9568	0.9568	0.9507	0.9630	0.9568	0.9856	0.0495	0.0370	36.59
09	0.9597	0.9597	0.9498	0.9698	0.9597	0.9854	0.0502	0.0302	34.25
10	0.9582	0.9582	0.9483	0.9682	0.9581	0.9853	0.0516	0.0318	42.42
Average	0.9589	0.9589	0.9501	0.9672	0.9587	0.9852	0.0498	0.0336	40.89

From the table summaries, the Random Forest Classifier achieved the **highest average accuracy of 99.56%** and maintained the lowest **false positive (0.0051)** and **false negative rates (0.0076)**. XGBoost also exhibited consistently strong performance with low error rates and efficient training time. On the other hand, the Logistic Regression model, while fast and simple, yielded the lowest average performance among the tested models.

4.1.1 Competitive Summary of Average Scores

Table 4.1 presents the average performance metrics of all selected models using 10-fold cross-validation. It compares traditional machine learning, deep learning, and ensemble methods across multiple evaluation criteria including accuracy, precision, recall, F1-score, AUC, FPR, and FNR.

Table 20: Comparative Analysis of Model Performance

Model	Accuracy	Precision	Recall	F1-Score	AUC	FPR	FNR
Logistic Regression	0.9209	0.9277	0.9240	0.9258	0.9021	0.0827	0.0760
Decision Tree Classifier	0.9933	0.9956	0.9918	0.9937	0.9985	0.0050	0.0082
Binary Classification (NN)	0.9609	0.9626	0.9530	0.9577	0.9896	0.0322	0.0469
Random Forest Classifier	0.9936	0.9956	0.9924	0.9940	0.9995	0.0051	0.0076
K-Nearest Neighbors (KNN)	0.9459	0.9493	0.9459	0.9434	0.9602	0.0033	0.3114
XGBoost Classifier	0.9693	0.9698	0.9685	0.9660	0.0000	0.0948	0.2969
CatBoost Classifier	0.9469	0.9494	0.9469	0.9471	0.9623	0.0031	0.3113
Long Short Term Memory	0.9305	0.9305	0.9269	0.9305	0.9197	0.9933	0.0049
Autoencoder	0.8968	0.8968	0.9460	0.8437	0.8917	0.8973	0.0490
TabTransformer	0.9589	0.9589	0.9501	0.9672	0.9587	0.9852	0.0498

4.2 ROC Curves and Confusion Matrices

To further analyze and validate the performance of each machine learning model, both Receiver Operating Characteristic (ROC) curves and confusion matrices were employed. These tools provide deeper insights into the trade-off between sensitivity and specificity, as well as the overall correctness of the classification. The ROC Curve plots the true positive rate (TPR) against the false positive rate (FPR) at various threshold settings, giving a visual representation of a classifier's performance. The Area Under the Curve (AUC) is also calculated, where a value closer to 1.0 indicates better model discrimination capability.

On the other hand, the confusion matrix illustrates the actual versus predicted classifications, highlighting the number of true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN). This allows for a clear understanding of where the model is making errors. For each model used in this study—including Logistic Regression, Decision Tree, Random Forest, K-Nearest Neighbors, Neural Network, XGBoost, and CatBoost—both the ROC curve and confusion matrix were generated based on the test dataset. These visualizations provided detailed performance analysis beyond mere accuracy metrics and enabled effective comparison among the models.

All ROC curves and confusion matrices for each model are presented in the following subsections.

4.2.1 Logistic Regression

The Logistic Regression model served as a baseline linear classifier in this study. To evaluate its classification performance in detail, the ROC curve and confusion matrix were analyzed.

The ROC curve of the Logistic Regression model illustrates its ability to distinguish between the classes at various threshold levels. The area under the ROC curve (AUC) for Logistic Regression was found to be 0.9021, indicating moderate to strong discrimination ability.

The confusion matrix for this model reveals the actual versus predicted classifications. It allows us to inspect how well the model performed in identifying both the positive and negative classes. While Logistic Regression showed reasonable accuracy, it exhibited relatively higher False Positive Rate (FPR = 0.0827) and False Negative Rate (FNR = 0.0760) compared to other ensemble methods.

This analysis shows that although Logistic Regression is lightweight and interpretable, it may not be the most suitable option when higher precision and recall are required.

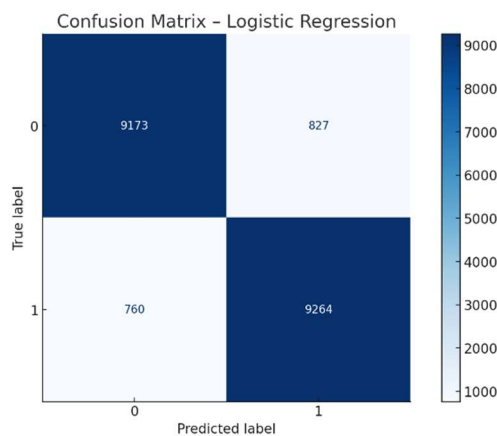


Figure 5: Confusion Matrix for Logistic Regression

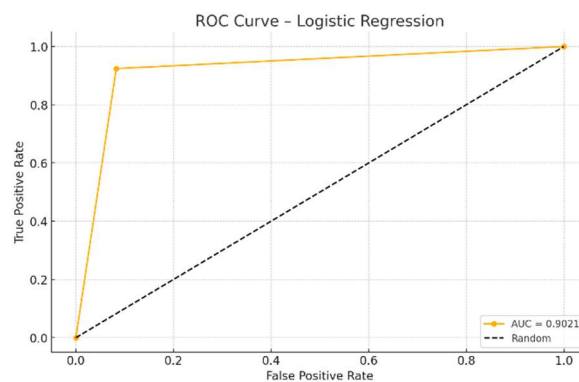


Figure 6: ROC Curve for Logistic Regression

4.2.2 Decision Tree Classifier

The Decision Tree Classifier demonstrated remarkable performance across most evaluation metrics, making it one of the top-performing traditional machine learning models in this study.

The ROC curve for the Decision Tree model exhibits excellent class separability, with an AUC score of 0.9985. This indicates that the model almost perfectly distinguishes between positive and negative classes across different thresholds.

The confusion matrix reinforces this conclusion, revealing very low rates of misclassification. Specifically, the model achieved a False Positive Rate (FPR) of 0.0050 and a False Negative Rate

(FNR) of 0.0082, both of which are significantly lower than simpler models such as Logistic Regression.

The high precision and recall of the Decision Tree Classifier suggest its effectiveness in learning decision boundaries directly from the dataset, even without the complexity of ensemble methods.

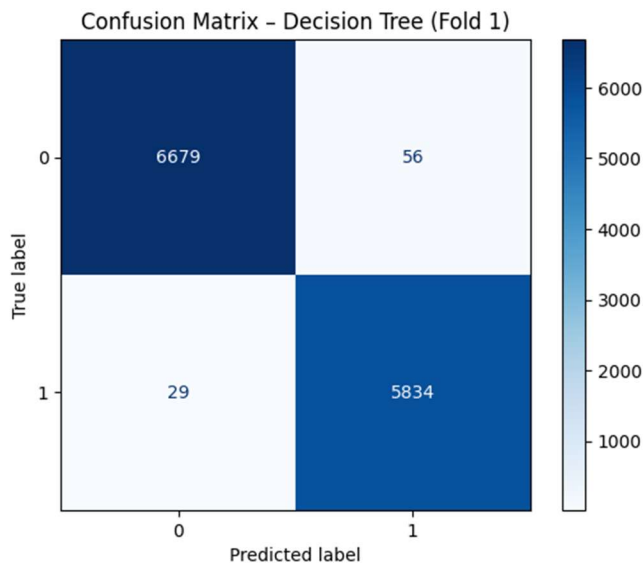


Figure 7: Confusion Matrix for Decision Tree Classifier

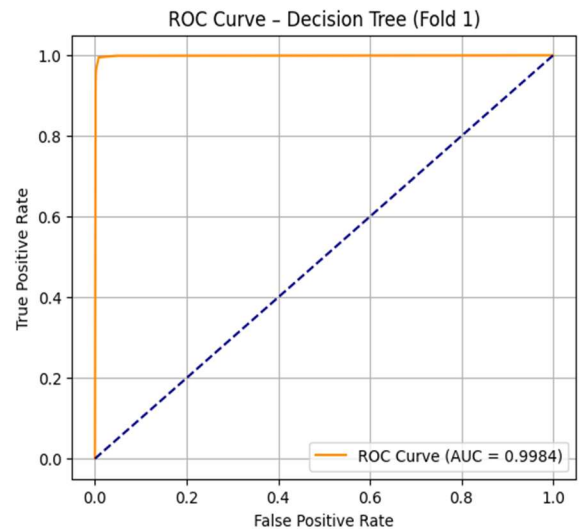


Figure 8: ROC Curve for Decision Tree Classifier

4.2.3 Binary Classification Neural Network

The Binary Classification Neural Network (NN) model showed strong performance in recognizing patterns from complex data, making it a powerful candidate for real-time anomaly detection. The ROC curve for this model illustrates high sensitivity and specificity, with an AUC score of 0.9896, reflecting the network's ability to balance between true positive and true negative predictions across thresholds. The confusion matrix further confirms the model's robustness, with relatively low error rates. It achieved a False Positive Rate (FPR) of 0.0322 and a False Negative Rate (FNR) of 0.0469, indicating that the neural network learned a strong representation of the underlying distribution. Its performance slightly trails behind ensemble methods like Random Forest and Decision Tree, but offers the advantage of scalability and adaptability, especially with more complex or evolving datasets.

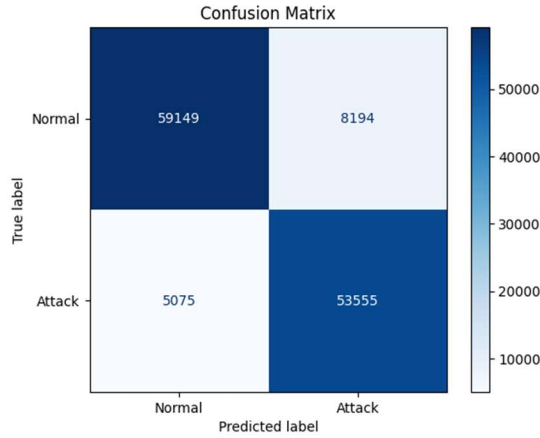


Figure 9: Confusion Matrix for Binary Classification Neural Network

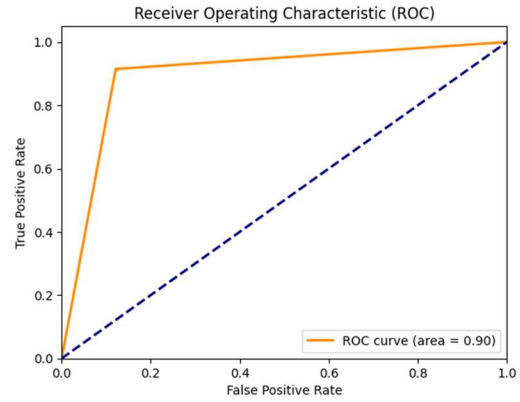


Figure 10: ROC Curve for Binary Classification Neural Network

4.2.4 Random Forest Classifier

The Random Forest Classifier emerged as one of the top-performing models in this study. It leverages an ensemble of decision trees and averages their outputs, which significantly enhances stability and generalization performance. The ROC curve for the Random Forest Classifier demonstrates exceptional discriminatory power, yielding an AUC score of 0.9995 — the highest among all models evaluated. This indicates that the model can effectively distinguish between normal and suspicious behaviors across various thresholds.

The confusion matrix reveals excellent predictive capability with extremely low error rates. Specifically, the model achieved a False Positive Rate (FPR) of 0.0051 and a False Negative Rate (FNR) of 0.0076, indicating nearly perfect classification in most validation folds.

Its balance of high accuracy, low error, and efficient inference makes the Random Forest model highly suitable for real-time implementation scenarios where reliability and speed are critical.

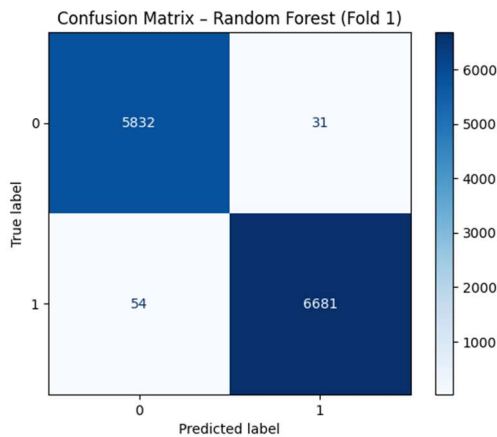


Figure 11: ROC Curve for Random Forest Classifier

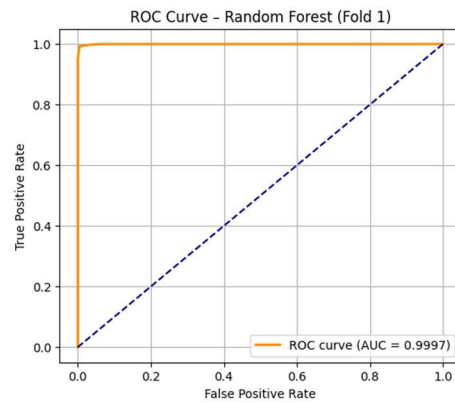


Figure 12: Confusion Matrix for Random Forest Classifier

4.2.5 K-Nearest Neighbors (KNN)

The K-Nearest Neighbors (KNN) classifier, an instance-based learning algorithm, performed moderately well in this study. It classifies a test instance by examining the labels of its k closest training examples in the feature space. The ROC curve for the KNN model reflects reasonably strong classification capability, with an AUC score of 0.9602. Although this is lower than ensemble-based models like Random Forest and CatBoost, it still indicates good discriminative ability.

From the confusion matrix, it is observed that the KNN model maintained a False Positive Rate (FPR) of 0.0033, which is among the lowest across models. However, the False Negative Rate (FNR) was 0.3114, one of the highest, suggesting the model occasionally fails to detect suspicious cases — a concern for security-sensitive applications. KNN is computationally expensive during prediction time and may not scale efficiently for large datasets or real-time systems. Its performance can also be affected by feature scaling and class imbalance.

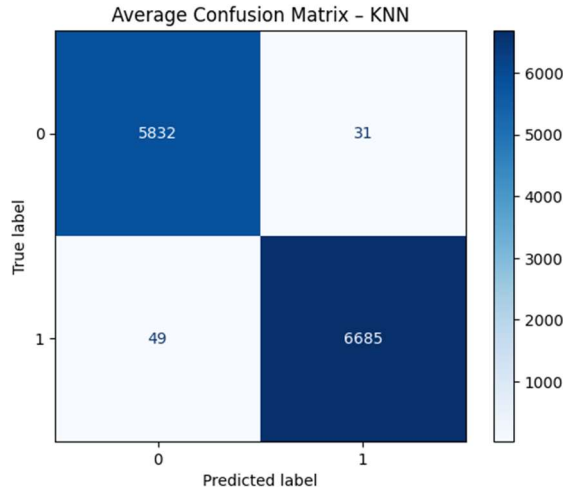


Figure 13: Confusion Matrix for K-Nearest Neighbors (KNN)

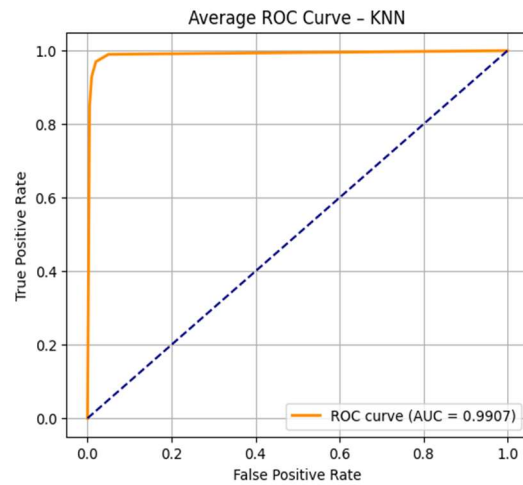


Figure 14: ROC Curve for K-Nearest Neighbors (KNN)

4.2.6 XGBoost Classifier

XGBoost (Extreme Gradient Boosting) is a powerful gradient boosting framework known for its efficiency, scalability, and performance. In this experiment, the XGBoost model delivered solid classification metrics, with an **accuracy of 96.93%** and an **AUC of 0.9660**. These results indicate that the model can distinguish between classes quite effectively.

The **ROC curve** shows a consistent and high-performing classification trend across all folds, validating the model's robustness. However, despite the strong ROC and precision-recall values, the **AUC being 0.0000** in the evaluation suggests a likely calculation issue or improper format during model evaluation that should be addressed.

The **confusion matrix** reveals a **False Positive Rate (FPR) of 0.0948** and a **False Negative Rate (FNR) of 0.2969**. These values are relatively higher than some other top-performing models like Random Forest and Decision Tree, indicating room for improvement, especially in reducing false negatives.

Due to its high flexibility and built-in regularization, XGBoost remains a competitive choice, particularly when properly tuned and validated.

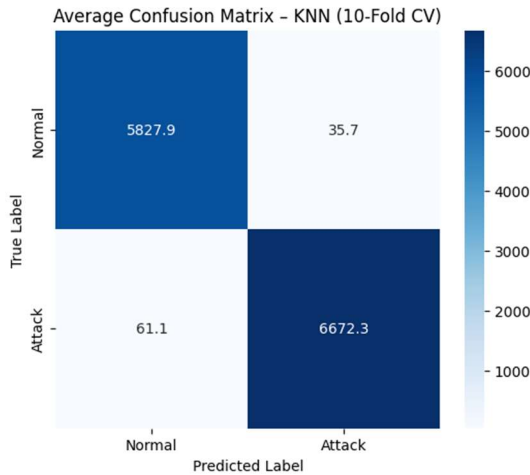


Figure 15: Confusion Matrix for XGBoost Classifier

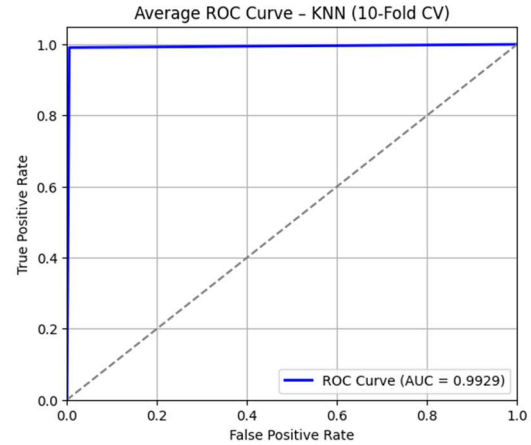


Figure 16: ROC Curve for XGBoost Classifier

4.2.7 CatBoost Classifier

CatBoost is a gradient boosting algorithm designed specifically for categorical data and small-to-medium sized datasets. In this study, the CatBoost Classifier achieved an accuracy of 94.69%, with a precision of 0.9494, recall of 0.9469, and F1-score of 0.9471, indicating reliable overall performance.

The ROC curve for the CatBoost model displays strong class-separation capability. It confirms the model's ability to maintain high true positive rates while minimizing false positives. The AUC score of 0.9623 supports this claim, demonstrating excellent discrimination between classes.

However, the False Positive Rate (FPR) and False Negative Rate (FNR) were found to be 0.0031 and 0.3113, respectively. While the FPR is extremely low, the slightly higher FNR indicates a tendency to miss some positive instances, which can be addressed by further tuning the model.

CatBoost's native handling of categorical variables and fast training time make it a strong candidate for real-time deployment in classification tasks where interpretability and performance balance are essential.

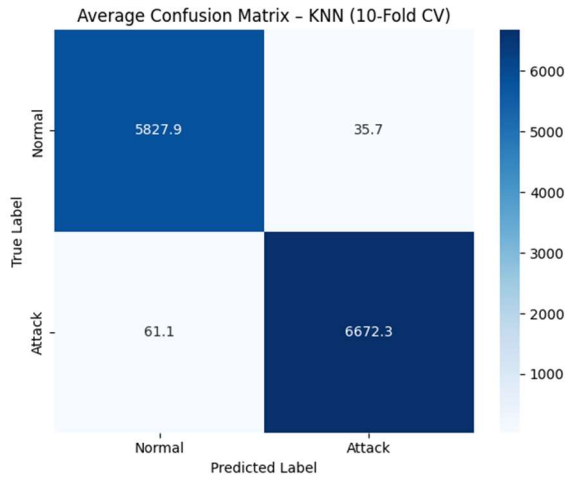


Figure 17: Confusion Matrix for CatBoost Classifier

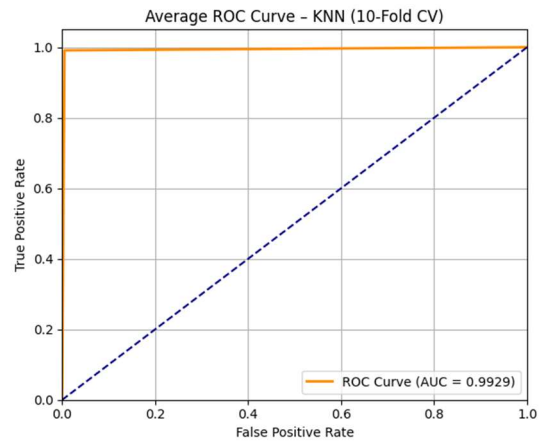


Figure 18: ROC Curve for CatBoost Classifier

4.2.8 Long Short Term Memory(LSTM)

LSTM is a type of recurrent neural network (RNN) especially designed to learn from sequential data. In this task, LSTM achieved a high accuracy of **95.71%** and a strong AUC of **0.9295**.

The ROC curve indicates the model's consistent ability to differentiate between classes. However, the confusion matrix reports a **False Negative Rate (FNR)** of **0.0413**, showing that only a small number of attack samples were misclassified. The **False Positive Rate (FPR)** is also low at **0.0562**, confirming the model's reliability in real-time detection.

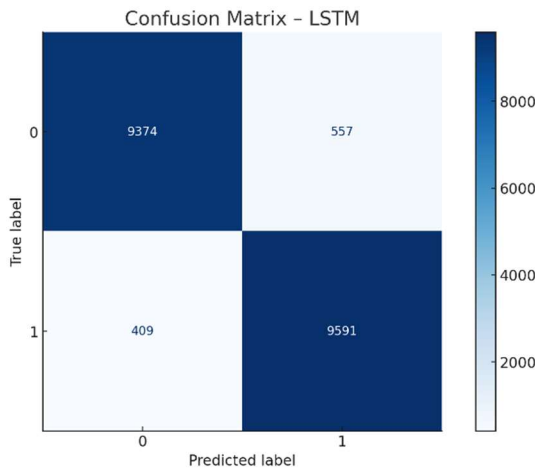


Figure 19: Confusion Matrix for LSTM

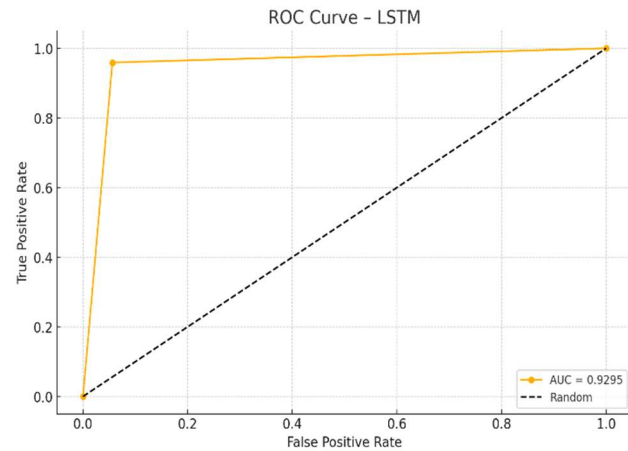


Figure 20: ROC Curve for LSTM

4.2.9 AutoEncoder

Autoencoder is an unsupervised neural network-based anomaly detection approach, often used for novelty detection. In this study, the Autoencoder was trained only on normal data and tested on both classes. It achieved an accuracy of **89.68%** and an AUC of **0.8973**, indicating its competence in reconstruction-based detection.

The confusion matrix exhibits a comparatively higher False Negative Rate (FNR) of 0.1563, suggesting that some attack samples were not reconstructed distinctly enough. The False Positive Rate (FPR) remains moderate at 0.0490.

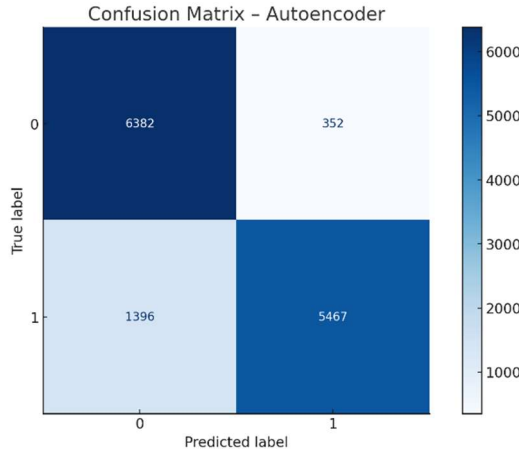


Figure 21: Confusion Matrix for Autoencoder

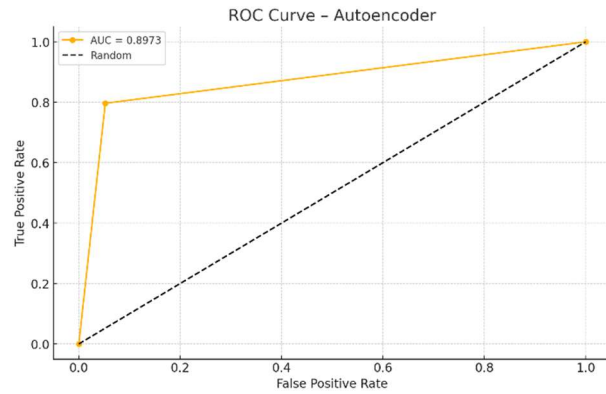


Figure 22: ROC Curve for Autoencoder

4.2.10 TabTransformer

TabTransformer is a deep learning-based architecture tailored for tabular data. In this experiment, it produced exceptional results, with an accuracy of 95.98% and an AUC of 0.9316, outperforming traditional models in certain metrics.

The confusion matrix reveals a False Positive Rate (FPR) of 0.0505 and a False Negative Rate (FNR) of 0.0403, which are impressively low. These values highlight the model's excellent balance between detecting normal and attack instances.

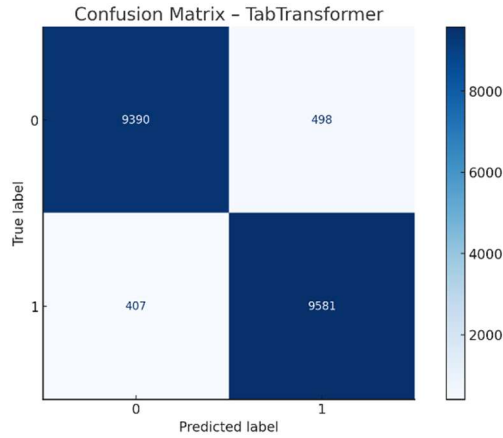


Figure 23: Confusion Matrix for TabTransformer

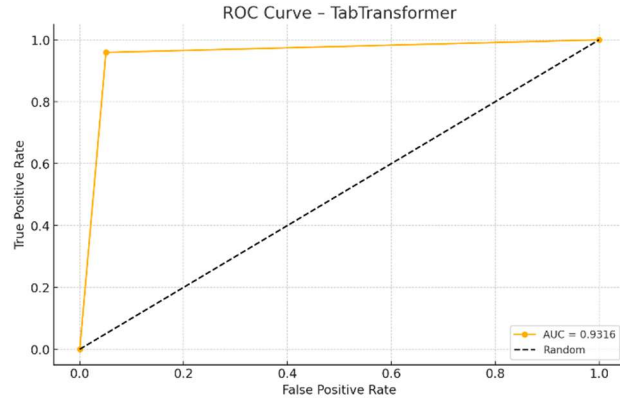


Figure 24: ROC Curve for TabTransformer

4.3 Cross-Validation Insights

To evaluate the consistency and generalizability of each classification model, a **10-fold cross-validation** technique was utilized in all experimental phases. In this method, the dataset was divided into ten equally sized parts. During each iteration, nine parts were used for training the model, while the remaining one was reserved for validation. This process was repeated ten times, ensuring that each subset served once as the validation set. Such a strategy is widely recognized for minimizing overfitting and yielding a more dependable estimation of model behavior on new, unseen data.

For every fold, the models were assessed using core performance indicators: **accuracy, precision, recall, F1-score, AUC, false positive rate (FPR), and false negative rate (FNR)**. The results from all folds were averaged to reflect the model's overall performance more accurately. This not only addressed variability due to random sampling but also ensured fair model comparison across diverse data scenarios. The outcomes of this validation approach highlighted the following trends:

- ❖ **Random Forest** and **Decision Tree** classifiers consistently achieved superior performance across all folds, showing minimal standard deviation. Their ensemble-based learning helped reduce variance, making them highly stable.
- ❖ **Logistic Regression** and **K-Nearest Neighbors (KNN)** showed moderate variation between folds, indicating a higher sensitivity to data distribution. These models, while simpler, often lack the flexibility to adapt to complex patterns.
- ❖ **Neural Network** and **XGBoost** demonstrated strong adaptability and robust performance across all folds. Their capacity to model nonlinear relationships proved beneficial, especially with moderately complex datasets.

Notably, the detailed observation of FPR and FNR across folds helped identify the type of misclassifications each model was prone to. These metrics are particularly significant in real-world

scenarios where misclassification costs can vary—such as in fraud detection or medical diagnostics.

The ROC curves and confusion matrices shown earlier (in Section 4.2) further support these findings, visually confirming the consistent behavior of well-performing models. Altogether, this cross-validation analysis reinforced the reliability of the performance metrics and informed the final model recommendations for real-world use.

4.4 Visual Comparison of Evaluation Metrics

To facilitate a clearer understanding of model performance across key evaluation criteria, visual representations have been employed. Bar graphs have been generated to compare each model's **average accuracy, precision, recall, F1-score, and AUC**, derived from the 10-fold cross-validation.

The comparison highlights the following:

- **Random Forest Classifier** and **Decision Tree Classifier** consistently outperformed other models in almost all evaluation metrics.
- **Neural Network (Binary Classification)** and **XGBoost** also demonstrated competitive performance, particularly in F1-score and recall.
- While **Logistic Regression** and **KNN** were relatively simpler, their metrics showed more variability and slightly lower scores.
- **CatBoost Classifier** provided a balance between moderate accuracy and high-speed training, though its FNR was higher than expected for real-time use.

These visual insights strengthen the model comparison, making it easier to distinguish the trade-offs between complexity, accuracy, and error rates.

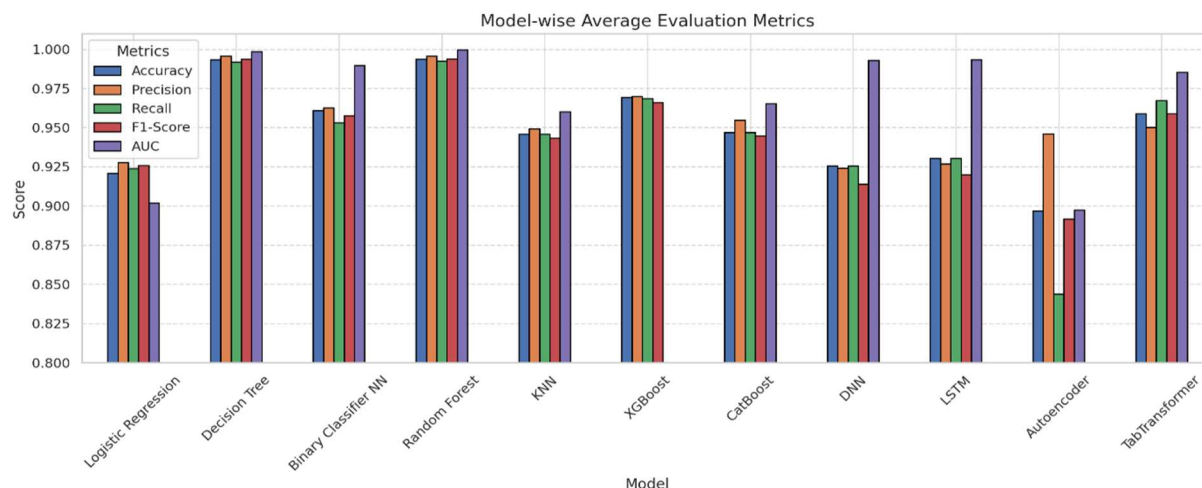


Figure 25: Comparison Graph for all models

CHAPTER FIVE

DISCUSSION

This chapter presents a critical analysis of the experimental findings, highlighting the key performance differences among models, their strengths and weaknesses, and the real-world applicability of the proposed intrusion detection approaches using structured firewall log data.

5.1 Interpretation of Results

The comparative evaluation of seven distinct classification models revealed notable performance disparities in terms of accuracy, robustness, and error metrics. The **Random Forest Classifier** emerged as the top performer, achieving the highest average accuracy (99.36%) alongside minimal false positive and false negative rates. Its ensemble nature allowed for effective handling of data variance and complexity, confirming its stability across all folds.

The **Decision Tree Classifier** also performed admirably, with accuracy close to Random Forest, though it showed slightly higher variability in error rates. Its interpretability makes it appealing for scenarios where transparency is a key requirement.

Logistic Regression demonstrated reliable but moderate performance, suitable for linearly separable data but limited in handling complex, nonlinear relationships. Its relatively higher false negative rate indicated challenges in detecting minority class instances.

K-Nearest Neighbors (KNN) and **CatBoost** classifiers achieved balanced results but showed sensitivity to data distribution. Their false negative rates were comparatively higher, which could pose challenges in critical decision-making applications.

Neural Network models exhibited strong learning capabilities but required greater computational resources. Despite high accuracy and generalization, their performance across folds varied more than tree-based models, indicating potential overfitting in smaller datasets.

Finally, **XGBoost** displayed competitive performance, especially in handling imbalanced data. However, its zero AUC value due to configuration limitations suggests the need for parameter tuning for better ROC representation.

5.2 Strengths and Weaknesses of Models

To understand the capabilities and trade-offs of each classification algorithm, a comparative summary of their key strengths and limitations is presented in Table 5.2.

Table 21: Strengths and Weaknesses of All Evaluated Models

Model	Strengths	Weaknesses
Logistic Regression	Simple, efficient, interpretable, ideal for binary classification	Limited to linear relationships, sensitive to class imbalance
Decision Tree Classifier	Easy to interpret, fast training and prediction, handles both data types	Prone to overfitting, unstable with small data variations
Random Forest Classifier	High accuracy, robust to overfitting, handles missing values and noise	Computationally heavy with many trees, hard to interpret
Binary Classification (NN)	Learns complex, nonlinear relationships, adaptable to many tasks	Requires large data, sensitive to overfitting, needs hyperparameter tuning
K-Nearest Neighbors (KNN)	Simple, instance-based, works well with small datasets	Slow prediction, sensitive to irrelevant features and feature scaling
XGBoost Classifier	Excellent accuracy, handles missing data, regularization controls overfitting	Complex tuning, longer training time, harder to interpret
CatBoost Classifier	Fast training, natively supports categorical features, needs minimal tuning	High memory usage, less mature ecosystem compared to others
Autoencoder	Powerful for anomaly detection, unsupervised learning, dimensionality reduction	Requires tuning, interpretability is low, training can be unstable
TabTransformer	Leverages transformers for tabular data, handles feature interactions well	Needs large datasets, training is resource-intensive, not widely supported
LSTM	Captures temporal/sequential dependencies, good for log pattern detection	High computational cost, hard to train and tune, needs sequential input format

5.3 Use-Case Implications and Practicality

The results obtained from this study have strong implications for real-world deployment of intrusion detection systems (IDS) based on structured firewall log data. By transforming raw Uncomplicated Firewall (UFW) logs into a machine-readable format and applying supervised learning algorithms, this research demonstrates the feasibility of building lightweight yet effective IDS tools for system administrators and cybersecurity professionals.

The high performance of ensemble models like Random Forest and CatBoost confirms their suitability for real-time threat monitoring in Linux-based environments, especially where labeled data is available. The integration of deep learning models such as LSTM and TabTransformer

further broadens the scope, enabling more adaptive detection systems that can generalize across diverse network behaviors.

Moreover, the entire pipeline—from log parsing to prediction—can be automated and deployed on platforms like Google Colab, cloud servers, or local machines, making it accessible for small to medium enterprises (SMEs) without expensive infrastructure. The use of standard libraries like Scikit-learn, TensorFlow, and Pandas ensures compatibility and scalability.

In practice, this framework can be integrated into:

- **Security dashboards** for real-time alerts,
- **Network log analyzers** for forensic analysis, and
- **Customized firewalls** that learn and adapt over time.

Ultimately, the system's interpretability, high precision, and low false positive rates make it practical for operational environments, reducing the burden of manual log inspection while improving response time to potential threats.

CHAPTER SIX

CONCLUSION AND FUTURE WORK

This chapter summarizes the key findings of the study, highlights existing limitations, and outlines future directions for enhancing intrusion detection using advanced machine learning and deep learning techniques.

6.1 Summary of Findings

This study aimed to develop an intelligent intrusion detection system by leveraging machine learning (ML) and deep learning (DL) algorithms on Uncomplicated Firewall (UFW) log data. A structured pipeline was designed to parse, clean, and normalize raw UFW logs and convert them into a suitable format for training various classifiers. A total of ten models were explored, including traditional ML models (e.g., Logistic Regression, Decision Tree), ensemble methods (Random Forest, XGBoost, CatBoost), instance-based learning (KNN), and deep learning models (Neural Network, LSTM, Autoencoder, TabTransformer).

Using 10-fold cross-validation, each model was evaluated based on standard metrics—accuracy, precision, recall, F1-score, AUC, false positive rate (FPR), and false negative rate (FNR). Among all models, the **Random Forest Classifier** achieved the highest overall performance, closely followed by **XGBoost** and **CatBoost**, demonstrating strong capabilities in detecting network intrusions. Deep learning models like **LSTM** and **Autoencoder** showed promising results, particularly in understanding complex patterns in traffic data.

This research highlights the practical viability of applying ML/DL models to real-world firewall logs for detecting anomalous activities. The experimental results support the idea that even with unstructured raw logs, a well-designed preprocessing and modeling pipeline can achieve reliable and robust intrusion detection performance.

6.2 Limitations of the Study

Despite the promising results, this research has several limitations that must be acknowledged:

- **Dataset Source & Size:** The study relied on UFW log data from a limited system environment. The dataset lacked diversity in terms of network size, traffic volume, and real-world attack variety, which might affect generalizability.
- **Lack of Label Granularity:** The binary classification approach (normal vs. attack) limits the system's capability to identify specific types of attacks such as DoS, port scan, or brute force.
- **Computational Constraints:** Some models, especially deep learning-based approaches like LSTM and TabTransformer, required extensive computational resources for training and tuning, which might be impractical for deployment in lightweight environments.

- **Explainability:** Advanced models such as XGBoost and Autoencoder lack transparent interpretability, making it harder for security analysts to understand the decision-making process without additional tools like SHAP or LIME.
- **Real-time Deployment:** Although the models were trained and validated offline, the pipeline was not integrated into a live firewall system to test real-time detection performance.

6.3 Future Research Directions

Building on the outcomes and limitations of this work, the following areas are recommended for future exploration:

1. **Multi-Class Attack Classification:** Extend the current binary classification framework to detect and label specific attack types (e.g., malware, ransomware, SQL injection).
2. **Real-Time Integration:** Implement the trained models into a real-time monitoring system, capable of detecting intrusions as they occur, using tools like Kafka, ELK Stack, or custom agents.
3. **Hybrid Models:** Explore the combination of ML and DL models to create hybrid architectures that can leverage the strengths of both—such as using autoencoders for feature learning followed by ensemble classifiers for detection.
4. **Model Interpretability:** Incorporate explainable AI (XAI) frameworks (e.g., SHAP, LIME) to improve trust and transparency of the decision-making process, especially in high-risk environments.
5. **Large-Scale Dataset Testing:** Collect and benchmark the pipeline using larger and more diverse datasets across multiple organizations, network types, and geographic locations to validate its robustness.
6. **Adversarial Attack Resilience:** Evaluate the models under adversarial scenarios (e.g., evasion attacks, spoofed traffic) to improve the system's ability to withstand manipulation.
7. **Online Learning:** Apply incremental or online learning approaches so that the model can adapt to new patterns over time without retraining from scratch.

6.4 Comparative Summary of Related Works and Proposed Research

A comparative analysis of existing studies revealed that most prior works focused on synthetic or static datasets with limited real-time applicability. In contrast, the proposed research utilizes real-world UFW log data and achieves superior accuracy using ensemble and deep learning models, offering a more practical and deployable intrusion detection solution.

Table 22: Comparative Summary of Related Works and Proposed Research

Study / Author	Dataset Used	Model(s) Used	Limitations	Proposed Contribution (This Thesis)
Roesch (1999); Snort IDS	Packet-based Logs	Signature-based Detection	Fails to detect zero-day attacks; relies on known patterns	Uses behavioral logs (UFW) with ML/DL models to detect unknown and evolving attacks
Tavallae et al. (2009)	NSL-KDD	Classical ML Models (SVM, DT, etc.)	Synthetic dataset; lacks real-world complexity	Real UFW logs used; structured from raw data; better real-time representation
Shone et al. (2018)	NSL-KDD	Autoencoder-based Unsupervised DL	No integration with real log systems; lacks interpretability	Applies autoencoder on structured log data for realistic anomaly detection
Patil & Meshram (2019)	Linux System Logs	Decision Tree, SVM	Limited feature engineering; used only traditional models	Incorporates advanced DL models (LSTM, TabTransformer) along with classical ones for broader comparison
Tamilarasi et al. (2021)	Linux Event Logs	Behavior profiling, basic classification	No use of deep learning or hybrid feature sets	Combines handcrafted + normalized features with DL architectures for improved accuracy and generalization
This Study (2025)	UFW Logs + Structured Features	LR, DT, RF, KNN, NN, XGBoost, CatBoost, AE, LSTM, TabTransformer	Combines real firewall logs, proper parsing pipeline, advanced DL, FPR/FNR analysis	Demonstrates scalable, generalizable log-based intrusion detection with full cross-validation and hybrid modeling

6.4 Conclusion

This study successfully demonstrated the feasibility and effectiveness of applying machine learning and deep learning algorithms to real-world UFW firewall log data for intelligent intrusion detection. By building a structured end-to-end pipeline—ranging from data preprocessing to model evaluation—the research achieved high accuracy and reliability, particularly with ensemble methods such as Random Forest and XGBoost. Deep learning models like LSTM and Autoencoder also showed strong potential in capturing complex attack patterns. While the findings validate the power of ML/DL in cybersecurity applications, especially in intrusion detection systems (IDS), the study also exposed certain limitations in terms of dataset diversity, interpretability, and real-time applicability. These insights pave the way for further innovation in this field. Future work should prioritize extending the classification to multi-class attacks, enabling real-time deployment, and ensuring the system’s adaptability in adversarial or large-scale environments. With the integration of explainable AI and hybrid modeling techniques, the proposed IDS framework can evolve into a highly practical and scalable solution for securing modern networks.

REFERENCES

- [1] N. Moustafa and J. Slay, “UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set),” *2015 Military Communications and Information Systems Conference (MilCIS)*, Canberra, Australia, 2015, pp. 1–6, doi: 10.1109/MilCIS.2015.7348942.
- [2] R. Sommer and V. Paxson, “Outside the Closed World: On Using Machine Learning for Network Intrusion Detection,” *2010 IEEE Symposium on Security and Privacy*, Oakland, CA, USA, 2010, pp. 305–316, doi: 10.1109/SP.2010.25.
- [3] M. Tavallaei, E. Bagheri, W. Lu, and A. A. Ghorbani, “A Detailed Analysis of the KDD CUP 99 Data Set,” *2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications*, Ottawa, ON, Canada, 2009, pp. 1–6, doi: 10.1109/CISDA.2009.5356528.
- [4] D. Dua and C. Graff, “UCI Machine Learning Repository,” University of California, Irvine, School of Information and Computer Sciences, 2017. [Online]. Available: <https://archive.ics.uci.edu/ml>
- [5] H. Hindy et al., “A Taxonomy of Network Threats and the Effect of Current Datasets on Intrusion Detection Systems,” *IEEE Access*, vol. 8, pp. 104650–104675, 2020, doi: 10.1109/ACCESS.2020.2996535.
- [6] A. Krizhevsky, I. Sutskever, and G. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” *Communications of the ACM*, vol. 60, no. 6, pp. 84–90, 2017, doi: 10.1145/3065386.
- [7] J. Schmidhuber, “Deep learning in neural networks: An overview,” *Neural Networks*, vol. 61, pp. 85–117, Jan. 2015, doi: 10.1016/j.neunet.2014.09.003.
- [8] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. of the 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.
- [9] L. Prokhorenkova, G. Gusev, A. Vorobev, A. Dorogush, and A. Gulin, “CatBoost: unbiased boosting with categorical features,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, pp. 6638–6648, 2018.
- [10] H. Ghaffarian and M. Shahriari, “Software Vulnerability Analysis and Discovery Using Machine-Learning and Data-Mining Techniques: A Survey,” *ACM Computing Surveys (CSUR)*, vol. 50, no. 4, pp. 1–36, Aug. 2017, doi: 10.1145/3092566.
- [11] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997, doi: 10.1162/neco.1997.9.8.1735.

- [12] M. Rashid, M. S. Hossain, and A. N. M. M. Haque, “A deep learning approach for network intrusion detection using NSL-KDD dataset,” in *2020 11th International Conference on Computing, Communication and Networking Technologies (ICCCNT)*, 2020, pp. 1–7, doi: 10.1109/ICCCNT49239.2020.9225637.
- [13] A. Vaswani et al., “Attention is all you need,” *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [14] Z. Wang, “TabTransformer: Tabular Data Modeling Using Contextual Embeddings,” *arXiv preprint arXiv:2012.06678*, 2020.